- API key validation middleware, RBAC enforcement

4.3 Ingestion Pipeline

- Request validation → deserialization → write path
- Synchronous ingestion with database-native aggregation.

4.4 Data Storage Layer

- PostgreSQL schema (ERD)
- TimescaleDB hypertables, retention policies, continuous aggregates
- Index strategy for query patterns

4.5 API Layer (Django REST Framework)

- Endpoint catalog with OpenAPI 3.0 specs
- Request/response schemas, error codes, pagination

4.6 Error Aggregation Service

- Parsing kernel/NVIDIA/Docker logs, deduplication, timestamping

5. DASHBOARD SPECIFICATION

5.1 UI Architecture (Django Templates + HTMX)

5.2 Page Specifications

- Rig list: columns, filters, sorting, virtualization
- Rig detail: tabs, chart components, data refresh strategy

5.3 HTMX Polling Design

- Endpoint contracts, partial templates, cache headers

5.4 Charting Library Integration

- Data aggregation queries, time-range handling, export

6. DATA MODEL & SCHEMA VERSIONING

6.1 Rig Inventory Schema (static data)

6.2 Metric Time-Series Schema (TimescaleDB)

6.3 Error/Event Schema

6.4 Audit Log Schema

6.5 Schema Versioning Protocol

- Backward/forward compatibility rules, migration triggers

7. SECURITY ARCHITECTURE

7.1 Threat Model (STRIDE analysis)

7.2 API Key Lifecycle Management

- Hashing (Argon2), rotation, revocation propagation

7.3 Transport Security (TLS 1.3, cert management)

7.4 Rate Limiting & Abuse Prevention

7.5 Audit Logging Requirements & Retention

7.6 Secret Storage & Configuration Security

8. OPERATIONAL ARCHITECTURE

8.1 Deployment Runbook

- Ubuntu VPS setup, systemd services, environment vars

8.2 Backup & Disaster Recovery

- pg_dump strategy, WAL archiving, restore testing cadence

8.3 Monitoring the Monitor

- Internal health endpoints, Prometheus metrics, alert rules

8.4 Logging Strategy (structured JSON, log rotation, correlation IDs)

8.5 Upgrade & Migration Procedures

- Agent rollout, server deploy, schema migrations

9. PERFORMANCE & SCALING ANALYSIS

9.1 Payload Size Estimates (per rig, per minute)

1. EXECUTIVE SUMMARY

This document defines the comprehensive, implementation-ready architecture for the **GPU Rig Monitoring Platform v1.0**. It translates the detailed business and operational requirements into a deterministic technical blueprint, guiding the development, deployment, and day-two operations of the system.

1.1 Purpose & Scope

1.1.1 System Purpose

The platform is a specialized, multi-user telemetry and monitoring dashboard designed explicitly for **GPU rigs serving AI/LLM workloads**. Unlike generic infrastructure monitors, this system provides deep visibility into the unique failure modes of AI servers, including VRAM fragmentation, PCIe throughput bottlenecks, GPU-attached process mapping, and Docker container instability.

It empowers fleet operators to assess real-time health, analyze historical performance trends, and inspect deep hardware/software states without relying on manual SSH access.

1.1.2 Architectural Scope

The system is bounded by three primary components operating within a strict single-VPS topology:

1. **Client Agent:** A lightweight, fault-tolerant Python script executed via `cron` every 60 seconds on target rigs. It collects extensive hardware, software, and AI-specific metrics, constructs a versioned JSON payload, and delivers it securely over HTTPS.
2. **Ingestion & Storage Server:** A Django-based application running on a single Ubuntu VPS. It utilizes Django REST Framework (DRF) for secure API ingestion, PostgreSQL for relational inventory/audit data, and TimescaleDB for high-performance time-series metric storage and aggregation.
3. **Hypermedia Dashboard:** A server-rendered web interface built with Django Templates and HTMX. It provides a highly responsive, low-complexity user experience with 30-second polling, eliminating the need for heavy Single Page Application (SPA) frameworks or WebSocket infrastructure.

1.1.3 Target Scale & Topology

The v1 architecture is explicitly designed for **vertical scaling on a single Ubuntu VPS**, targeting a ceiling of **~1,000 rigs** reporting at 1-minute intervals. This constraint intentionally avoids the operational overhead of distributed microservices, message brokers, or multi-region clustering, ensuring the platform remains highly maintainable and cost-effective for its initial production lifecycle.

1.2 Non-Goals & Out-of-Scope (v1)

To ensure a focused, shippable, and stable first release, the following capabilities are explicitly excluded from the v1 architecture. These boundaries prevent scope creep and maintain the simplicity of the single-VPS deployment model.

Category	Out-of-Scope Item	Architectural Rationale
Communication	WebSockets / Server-Sent Events (SSE)	Adds persistent connection overhead and state-management complexity. HTMX 30s polling is sufficient for minute-level telemetry and drastically simplifies the Nginx/Gunicorn stack.
Data Ingestion	Full Log Aggregation (e.g., ELK, Loki)	Storing full `journalctl` or application logs transforms the system into a heavy log-management platform. v1 strictly limits error tracking to the *latest deduplicated critical errors* with timestamps.
Operations	Active Alerting & Notifications	No PagerDuty, email, or Slack webhook integrations. v1 relies on visual dashboard indicators (Stale/Offline badges, redlining metrics) for human-in-the-loop operations.
Topology	Distributed Deployment / Clustering	No Kubernetes, no separate worker nodes, no Celery/RabbitMQ queues. Background tasks are handled by native `systemd` timers and TimescaleDB continuous aggregates.
Extensibility	Public API / Third-Party Webhooks	The REST API is strictly internal, secured by user-scoped API keys for agent ingestion and session cookies for the dashboard. No OAuth2 provider or public developer portal.
Remediation	Remote Command Execution	The agent is strictly **read-only**. It cannot execute shell commands, restart services, or modify rig configurations remotely, mitigating severe security risks.

1.3 Success Metrics & KPIs

The architecture is engineered to meet strict performance, reliability, and operational budgets. The success of the v1 implementation will be measured against the following Key Performance Indicators (KPIs) during load testing (Sec 10) and production operation.

1.3.1 Ingestion & Pipeline Reliability

Metric	Target	Validation Method
Payload Acceptance Rate	`> 99.9%`	Ratio of `200 OK` / `202 Accepted` vs `4xx/5xx` errors under load.
Idempotency Accuracy	`100%`	Zero duplicate rows created in TimescaleDB during simulated network retries.
Agent Overhead	`< 2% CPU`, `< 50MB RAM`	Measured via `psutil` on the host rig during the 60-second cron execution window.
Network Footprint	`< 2.0 KB` per payload	Average gzip-compressed payload size over a 24-hour period.

1.3.2 Performance & Latency Budgets

Metric	Target	Validation Method
Ingest API Latency (p95)	`< 200ms`	Measured at the Gunicorn worker level during 1,000-rig Locust load tests.
Dashboard Chart Load	`< 500ms`	Time from HTMX trigger to Chart.js render for a 24-hour historical view (querying continuous aggregates).
Fleet List Polling Swap	`< 100ms`	Time for Nginx to serve and browser to morphdom-swap the 50-row fleet table partial.

1.3.3 Operational & Resource Efficiency

Metric	Target	Validation Method
VPS CPU Saturation	`< 60%` average	Measured via `netdata`/`htop` on the 4-vCPU VPS at peak 1,000-rig burst (50 RPS).
Database Storage Growth	`< 5 GB / day`	Raw hypertable size + indexes at 1,000 rigs, prior to 14-day compression/drop policies.
Time-to-First-	`< 2`	Time from executing the agent install script to the rig appearing as

Metric	Target	Validation Method
Payload	`minutes`	"Online" on the dashboard.
Backup Restore Validity	`100% success`	Monthly automated CI/CD pipeline that provisions a dummy VPS, restores the latest `pg_dump`, and passes Django `check --deploy`.

2. SYSTEM ARCHITECTURE OVERVIEW

This section defines the structural foundation, deployment topology, data movement patterns, and security zoning for the GPU Rig Monitoring Platform. It translates the high-level requirements into concrete, implementation-ready specifications that will drive the agent, server, database, and dashboard designs in subsequent sections.

2.1 High-Level Component Diagram (C4 Level 2)

The system follows a **hub-and-spoke telemetry model** with three primary container boundaries: Client Agent, Ingestion/API Server, and Dashboard/UI. All components communicate over HTTPS or internal loopback.

```
graph TD
    subgraph External [RIG FLEET (Untrusted)]
        A[Python Client Agent] -->|HTTPS POST /api/v1/ingest/| B[Nginx Reverse Proxy]
    end

    subgraph VPS [SINGLE UBUNTU VPS (Trusted)]
        B -->|HTTP/1.1| C[Django + DRF App]
        C -->|TCP/5432| D[(PostgreSQL + TimescaleDB)]
        C -->|Render/Query| E[HTMX Dashboard UI]
        F[User Browser] -->|HTTPS GET/POST + HTMX Polling| B
        B -->|HTTP| E
        E -->|AJAX/Fetch| C
    end

    style A fill:#f9f,stroke:#333,stroke-width:2px
    style B fill:#bbf,stroke:#333,stroke-width:2px
    style C fill:#bfb,stroke:#333,stroke-width:2px
```

style D fill:#fbb,stroke:#333,stroke-width:2px

style E fill:#ff9,stroke:#333,stroke-width:2px

style F fill:#fff,stroke:#333,stroke-width:2px

Component Responsibilities

Component	Technology	Responsibility	Implementation Boundary
Client Agent	Python 3.10+, `psutil`, `pynvml`, `requests`	Collects hardware/software metrics, constructs versioned JSON payload, handles retry/idempotency, runs via `cron`	Runs on customer rigs. No privileged escalation beyond `sudo` for SMART/NVIDIA queries if needed.
Nginx Edge	Nginx	TLS termination, request routing, baseline rate limiting, static asset serving, HTMX polling endpoint proxy	Binds to `0.0.0.0:443`, `0.0.0.0:80` (redirects to HTTPS). Internal proxy to app on `127.0.0.1:8000`.
Django App	Django 5.x, DRF, Gunicorn	Auth middleware, schema validation, payload routing, RBAC enforcement, dashboard rendering, HTMX partial responses	Sync workers for predictable DB connection handling. DRF for `/api/v1/ingest/`, standard views for `/dashboard/`.
PostgreSQL + TimescaleDB	PG 16, TimescaleDB 2.14+	Relational storage (users, rigs, keys, tags, audit), hypertable time-series storage, continuous aggregates, retention policies	Binds to `127.0.0.1:5432`. PgBouncer recommended for connection pooling at scale.
Dashboard UI	Django Templates + HTMX 2.0	Server-rendered pages, 30s polling swaps, chart data endpoints, tag filtering, rig detail tabs	No SPA framework. Progressive enhancement via `hx-get`, `hx-swap`, `hx-trigger`.

2.2 Deployment Topology

Infrastructure Baseline

- **OS:** Ubuntu 22.04/24.04 LTS

- **Compute Target:** 4–8 vCPU, 16–32 GB RAM, 500 GB+ NVMe SSD (scales vertically to ~1,000 rigs)
- **Network:** Single public IPv4, DNS A record (`monitor.example.com`)
- **TLS:** Let's Encrypt (`certbot`), auto-renew via systemd timer

Service & Port Matrix

Service	Bind Address	Port	Protocol	Purpose	Access Control
Nginx	`0.0.0.0`	80	HTTP	HTTPS redirect	Public
Nginx	`0.0.0.0`	443	HTTPS	Ingress for agents & dashboard	Public (TLS enforced)
Gunicorn	`127.0.0.1`	8000	HTTP	WSGI app server	Internal only
PostgreSQL	`127.0.0.1`	5432	TCP	Primary datastore	Internal only, SCRAM-SHA-256
PgBouncer (optional)	`127.0.0.1`	6432	TCP	Connection pooling	Internal only
SSH	`0.0.0.0`	22	TCP	Admin access	

Firewall Rules (`ufw`)

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp    # SSH (restrict to admin IPs in prod)
ufw allow 80/tcp    # HTTP redirect
ufw allow 443/tcp   # HTTPS
ufw enable
```

Process Management & Isolation

Service	Systemd Unit	User/Group	Restart Policy	Notes
`nginx.service`	`nginx`	`root`/`www-data`	`on-failure`	Handles TLS, routing, static files
`gunicorn.service`	`monitoring`	`monitoring`	`on-failure`	Runs Django app, 4–8 workers
`postgresql.service`	`postgres`	`postgres`	`on-failure`	TimescaleDB extension enabled
`cron` (backups)	`systemd.timer`	`root`	`always`	`pg_dump` + WAL archiving

Implementation Note: The Django app runs under a dedicated `monitoring` OS user with `no-login` shell. The DB user has `NOLOGIN`, app connects via `APP_DB_USER` with `SELECT/INSERT/UPDATE` on specific schemas only.

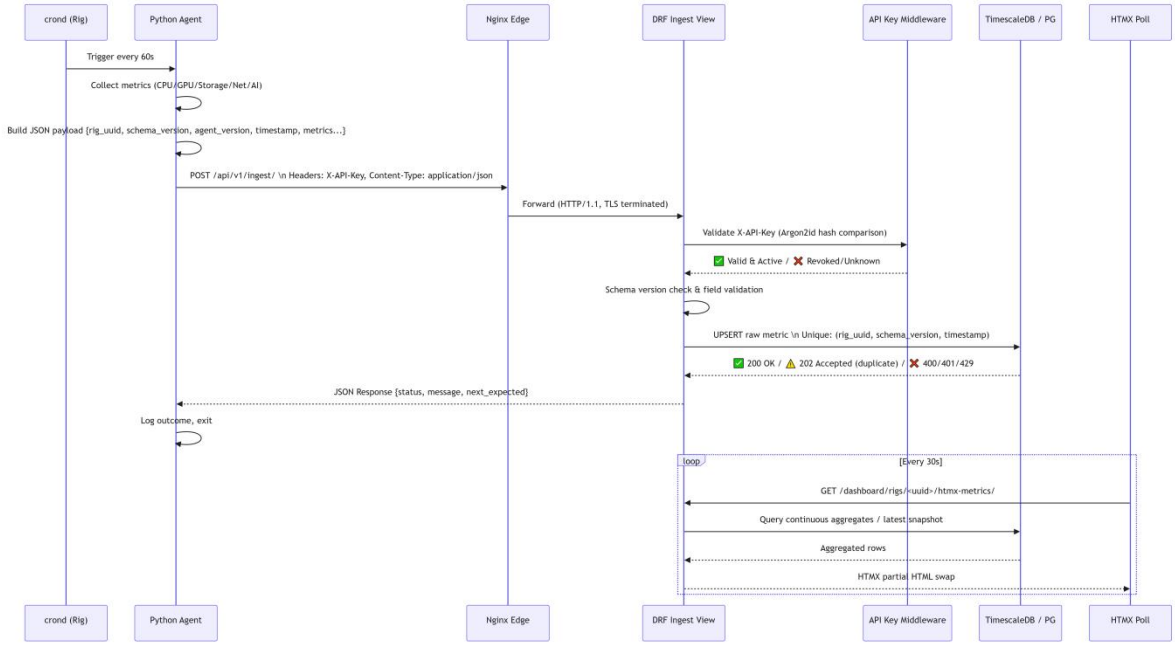
2.3 Data Flow Diagrams

2.3.1 Agent Ingestion Sequence

```
Mermaid

sequenceDiagram
    participant Cron as crond (Rig)
    participant Agent as Python Agent
    participant Nginx as Nginx Edge
    participant DRF as DRF Ingest View
    participant Auth as API Key Middleware
    participant DB as TimescaleDB / PG
    participant Dashboard as HTMX Poll

    Cron->>Agent: Trigger every 60s
    Agent->>Agent: Collect metrics (CPU/GPU/Storage/Net/AI)
    Agent->>Agent: Build JSON payload {rig_uuid,
schema_version, agent_version, timestamp, metrics...}
    Agent->>Nginx: POST /api/v1/ingest/ \n Headers: X-API-Key,
Content-Type: application/json
    Nginx->>DRF: Forward (HTTP/1.1, TLS terminated)
    DRF->>Auth: Validate X-API-Key (Argon2id hash comparison)
    Auth-->>DRF:  Valid & Active /  Revoked/Unknown
    DRF->>DRF: Schema version check & field validation
    DRF->>DB: UPSERT raw metric \n Unique: (rig_uuid,
schema_version, timestamp)
    DB-->>DRF:  200 OK / ⚠ 202 Accepted (duplicate) / 
400/401/429
    DRF-->>Agent: JSON Response {status, message,
next_expected}
    Agent->>Agent: Log outcome, exit
    loop Every 30s
        Dashboard->>DRF: GET /dashboard/rigs/<uuid>/htmx-
metrics/
        DRF->>DB: Query continuous aggregates / latest
snapshot
        DB-->>DRF: Aggregated rows
        DRF-->>Dashboard: HTMX partial HTML swap
    End
```


crond (Rig)
Python Agent
Nginx Edge
DRF Ingest View
API Key Middleware
TimescaleDB / PG
HTMX Poll
Trigger every 60s
Collect metrics (CPU/GPU/Storage/Net/AI)
Build JSON payload {rig_uuid, schema_version, agent_version, timestamp, metrics...}
POST /api/v1/ingest/ \n Headers: X-API-Key, Content-Type: application/json
Forward (HTTP/1.1, TLS terminated)
Validate X-API-Key (Argon2id hash comparison)
✅ Valid & Active / ❌ Revoked/Unknown
Schema version check & field validation
UPSERT raw metric \n Unique: (rig_uuid, schema_version, timestamp)
✅ 200 OK / ⚠️ 202 Accepted (duplicate) / ❌ 400/401/429
JSON Response {status, message, next_expected}
Log outcome, exit
loop
[Every 30s]
GET /dashboard/rigs/<uuid>/htmx-metrics/
Query continuous aggregates / latest snapshot
Aggregated rows
HTMX partial HTML swap
crond (Rig)
Python Agent
Nginx Edge
DRF Ingest View
API Key Middleware
TimescaleDB / PG
HTMX Poll

2.3.2 Ingestion Contract & Payload Handling

Step	Requirement	Implementation Detail
Auth	API key on every request	`X-API-Key` header. Server stores `pbkdf2_sha256` or `argon2id` hash. Constant-time comparison via Django's `check_password()`.
Idempotency	Avoid duplicate storage	Composite `UNIQUE` constraint: `(rig_uuid, schema_version, timestamp)`. Postgres `ON CONFLICT DO NOTHING` or `UPDATE`. Returns `200` on new, `202` on duplicate.
Retry Logic	Safe failure recovery	Agent implements exponential backoff (1s → 2s → 4s → max 30s, 5 attempts). Non-blocking: respects `cron` schedule, queues to in-memory buffer if needed.
Partial Failures	Tolerate metric gaps	Agent wraps collectors in `try/except`. Missing fields omitted. Server uses `COALESCE` or ignores nulls. No `400` for partial payloads.
Payload Size	Network efficiency	Typical payload: `8–15 KB` JSON. Gzip compression enabled at Nginx & agent (`requests compress=True`).

2.4 Trust Boundaries & Security Zones

The architecture enforces **defense-in-depth** across four explicit trust zones. Every data transition crosses a validation boundary.

2.4.1 Zone Mapping

Zone	Trust Level	Components	Enforcement Mechanism
Z1: External Fleet	Untrusted	Client Agents, Public Internet	TLS 1.3, API key auth, rate limiting, payload schema validation
Z2: Edge/DMZ	Semi-Trusted	Nginx, certbot, UFW	TLS termination, WAF rules (ModSecurity optional), connection limits, request size caps (`client_max_body_size 2M`)
Z3: Application	Trusted	Django, Gunicorn, DRF, HTMX views	Session auth, CSRF tokens, RBAC middleware, parameterized queries, audit logging, input sanitization
Z4: Data Layer	Highly Trusted	PostgreSQL, TimescaleDB, backups	Network isolation (`127.0.0.1`), least-privilege roles, row-level security (optional), encrypted backups, immutable audit tables

2.4.2 Boundary Validation Matrix

Boundary	Threat	Mitigation
Z1 → Z2	MITM, Replay, Flood	Mandatory HTTPS, HSTS, Nginx `limit_req_zone`, TLS 1.3 only
Z2 → Z3	Auth Bypass, Payload Injection	DRF `authentication_classes`, `permission_classes`, strict JSON schema validation (`pydantic` or DRF serializers)
Z3 → Z4	SQLi, Data Leakage, Privilege Escalation	ORM only, no raw SQL except TimescaleDB maintenance scripts, separate DB roles (`app_reader`, `app_writer`), audit triggers
Dashboard User → Z3	XSS, CSRF, Session Hijacking	Django `CsrfViewMiddleware`, `Secure`/`HttpOnly`/`SameSite=Lax` cookies, HTMX auto-escapes, CSP headers

2.4.3 API Key Lifecycle & Hashing

- **Generation:** Server creates 32-byte random hex string → hashes with `argon2id` (memory=65536, iterations=3, parallelism=4) → stores hash + salt in `auth_apikey` table → returns plaintext key **once** to user.
- **Validation:** On ingest, Django uses `argon2-cffi` constant-time verify. Never logs plaintext keys.
- **Rotation/Revocation:** `is_active` flag + `revoked_at` timestamp. Revocation propagates on next request (no cache). Audit log entry created.

2.4.4 Rate Limiting Strategy

Layer	Algorithm	Threshold	Action on Exceed
Nginx	Fixed Window (leaky bucket)	10 req/s per IP	`429 Too Many Requests`
DRF	Sliding Window (per API key)	2 req/min per rig	`429` + backoff hint in response
Global	Connection limit	500 concurrent	Nginx `worker_connections` tuned

□ Transition to Next Section

This overview establishes the **structural contract** for the system:

- All telemetry flows through a validated, idempotent ingestion pipeline.
- The single VPS topology relies on vertical scaling, connection pooling, and TimescaleDB aggregation to handle ~1,000 rigs at 1-minute intervals.
- Security boundaries are explicitly enforced at TLS, app, and DB layers with hashed credentials and RBAC.

3. CLIENT AGENT SPECIFICATION

This section defines the exact runtime environment, configuration schema, metric collection methodology, payload structure, transport behavior, and execution guardrails for the client agent. It translates the requirements into deterministic implementation rules that ensure reliability, idempotency, and graceful degradation across heterogeneous GPU rigs.

3.1 Runtime Environment & Dependencies

Category	Specification	Rationale
Python Version	`3.10+` (matches Ubuntu 22.04/24.04 default)	Ensures native `match/case`, `zoneinfo`, and `asyncio` stability
Installation Path	`/opt/monitoring-agent/`	Standardized, avoids user-home conflicts
Virtual Env	`/opt/monitoring-agent/venv/`	Isolates dependencies from system packages
Core Python Deps	`psutil==5.9.8`, `pynvml==11.5.0`, `py-cpuinfo==9.0.0`, `requests==2.31.0`, `docker==7.0.0`, `pyyaml==6.0.1`	Mature, actively maintained, low overhead
System Binaries	`smartmontools`, `nvme-cli`, `lm-sensors`, `jq` (optional for debug)	Required for SMART, NVMe logs, and CPU temp fallbacks
Execution User	`monitoring-agent` (system, no-login)	Least-privilege baseline. `sudo` granted only for specific telemetry commands
Sudoers Policy	`/etc/sudoers.d/monitoring-agent` `monitoring-agent ALL=(root) NOPASSWD: /usr/sbin/smartctl, /usr/bin/nvme, /bin/journalctl, /usr/bin/systemctl list-units`	Prevents privilege escalation while enabling disk/log access

3.2 Configuration Management

3.2.1 Config File Spec

- **Path:** `/etc/monitoring-agent/config.yaml`
- **Permissions:** `0600`, owned by `root:monitoring-agent`
- **Format:**

```
yaml

rig_uuid: "auto"           # Generates uuid4() on first run if "auto"
api_key: ""                # Plaintext during setup, read-only by
agent
server_endpoint: "https://monitor.example.com"
expected_gpu_count: 0      # 0 = auto-detect; >0 = enforce validation
collection_timeout_s: 45   # Hard limit per run
retry_attempts: 3
debug_mode: false
```

3.2.2 UUID Lifecycle

- Generated via `uuid.uuid4()` if `rig_uuid: "auto"` or missing.
- Persisted immediately to config file on disk.
- **Never regenerated** across reboots, OS reinstalls, or agent upgrades.
- Ownership is bound server-side on first successful ingest (immutable per RBAC rules).

3.2.3 Config Validation & Reload

- Agent validates required fields (`api_key`, `server_endpoint`) at startup.
- Exits with code `2` + log error if invalid.
- Optional `SIGHUP` handler reloads config without killing cron cycle.

3.3 Metric Collection Modules

Each module is wrapped in an isolated `try/except` block. Failures are logged locally and omitted from the payload. The agent **never aborts** due to a single module failure.

Module	Primary Data Source	Fallback / Graceful Degradation	Output Fields	Error Handling
CPU	`py-cpuinfo.get_cpu_info()`, `psutil.cpu_percent()`, `psutil.sensors_temperatures()`	Skip temp if `sensors` unavailable; fallback `psutil.cpu_count()` for core counts	`model`, `vendor`, `arch`, `sockets`, `physical_cores`, `logical_cores`, `load_avg`, `utilization_`	Log warning, omit `temp_c` if `None`

Module	Primary Data Source	Fallback / Graceful Degradation	Output Fields	Error Handling
			`pct`, `temp_c`	
Memory	`psutil.virtual_memory()`, `psutil.swap_memory()`	N/A (always available)	`total_bytes`, `used_bytes`, `free_bytes`, `cached_bytes`, `swap_used_bytes`, `swap_total_bytes`	None
Motherboard	`/sys/class/dmi/id/board_{vendor,product,bios_version}`	Fallback to `"unknown"` if files unreadable	`manufacturer`, `model`, `bios_version`	Skip if permission denied
Storage	`psutil.disk_partitions()`, `psutil.disk_usage()` + `sudo smartctl -a` / `sudo nvme smart-log`	Skip SMART/NVMe if `sudo` fails or device unsupported	`device`, `model`, `serial`, `capacity_bytes`, `usage_pct`, `mount_state`, `smart_health`, `temp_c`, `nvme_wear_level`, `nvme_media_errors`	Catch `subprocess.CalledProcessError`, log, omit health fields
Network	`psutil.net_if_addrs()`, `psutil.net_io_counters()`, `/sys/class/net/<iface>/speed`	Skip link speed if `speed` file returns `-1`	`interface`, `ipv4`, `link_speed_mbps`, `rx_bytes`, `tx_bytes`, `rx_errors`, `tx_errors`	Skip unphysical/loopback interfaces
GPU	`pynvml` (NVIDIA Management Library)	Skip unsupported metrics (e.g., fan speed on passive cards)	`uuid`, `pci_bus_id`, `model`, `mem_total_mb`, `mem_used_mb`, `mem_free_mb`, `mem_util_pct`, `gpu_util_pct`	Handle `NVMLError_NotSupported` → return `null`

Module	Primary Data Source	Fallback / Graceful Degradation	Output Fields	Error Handling
			, `temp_c`, `fan_speed_pct`, `power_draw_w`, `power_limit_w`, `pcie_gen`, `pcie_width`, `pcie_throughput_gbps`, `processes: [{pid, name, mem_mb}]`	
AI/LLM Heuristics	Derived from `pynvml` + process list	N/A	`fragmentation_risk: "low"	"medium"	"high"`	
Docker	`docker` Python SDK (`docker.from_env()`)	Skip if daemon unreachable or socket permissions denied	`containers: [{name, image, status, restart_count, uptime_seconds}]`	Log warning, omit array if failed
Software/OS	`platform`, `psutil.boot_time()`, `timedatectl`, `nvidia-smi --query`, `docker version`	Use `Unknown` for missing fields	`hostname`, `os_distro`, `kernel`, `uptime_s`, `ntp_sync: bool`, `nvidia_driver`, `cuda_version`, `docker_version`	Partial failure tolerated
Error Aggregation	`journalctl -p err..crit --since "5 min ago"`, `systemctl --failed`	Truncate to last 20 entries, deduplicate by message hash	`errors: [{source: "kernel"	"nvidia"	"docker"	"systemd", timestamp, message, count}]`	

3.3.1 GPU Count Validation Logic

1. Read `expected_gpu_count` from config.
2. If `> 0`, query `pynvml.nvmlDeviceGetCount()`.
3. Set `gpu_mismatch = (actual != expected)`.
4. Emit warning-level log + flag in payload. Server-side UI highlights mismatch in red.

3.4 Payload Construction & Serialization

3.4.1 JSON Schema (v1.0)

```
Python

{
  "rig_uuid": "string (UUIDv4)",
  "schema_version": "string (constant: '1.0')",
  "agent_version": "string (semver: '1.0.0')",
  "timestamp": "string (ISO 8601, UTC, e.g., '2024-05-
20T14:32:00Z')",
  "inventory": {
    "cpu": { ... },
    "memory": { ... },
    "motherboard": { ... },
    "storage": [ { ... } ],
    "network": [ { ... } ],
    "gpus": [ { ... } ]
  },
  "metrics": {
    "cpu": { ... },
    "memory": { ... },
    "storage": [ { ... } ],
    "network": [ { ... } ],
    "gpus": [ { ... } ],
    "ai_processes": [ { ... } ],
    "docker_containers": [ { ... } ]
  },
  "software": { ... },
  "errors": [ { ... } ]
}
```

3.4.2 Serialization Rules

- **Nullability:** Missing/failed metrics are explicitly `null`, not omitted, unless the entire array/object is empty.
- **Types:** Strict typing. `null` allowed where specified. No implicit string coercion.
- **Compression:** Agent applies `gzip` if uncompressed payload `> 8 KB`. Sets `Content-Encoding: gzip`.
- **Integrity:** No cryptographic signing in v1. Relies on HTTPS + server-side schema validation.
- **Size Budget:** Target `< 20 KB` compressed. Payloads exceeding `2 MB` are rejected by Nginx (`client_max_body_size`).

3.5 Transport Layer & Delivery Contract

3.5.1 HTTP Client Configuration

```
Python

session = requests.Session()
session.headers.update({
    "Content-Type": "application/json",
    "X-API-Key": config.api_key,
    "User-Agent": f"rig-monitor-agent/{agent_version}"
})
timeout = (3.0, 10.0)  # (connect, read)
```

3.5.2 Retry & Backoff Algorithm

- **Library:** `tenacity` or manual implementation.
- **Strategy:** Exponential backoff with jitter.
- **Sequence:** `1s → 2s → 4s` (max 3 attempts per cron cycle).
- **Jitter:** `±20%` random delay to prevent thundering herd.
- **Abort Condition:** If `collection_timeout_s` (45s) is exceeded, agent kills pending requests and exits with code `1`.

3.5.3 Idempotency & Server Contract

- **Agent Role:** Stateless. Does not track delivery state.
- **Server Role:** Enforces idempotency via `UNIQUE(rig_uuid, schema_version, timestamp)` + `ON CONFLICT DO UPDATE`.
- **Response Codes:**
  - `200 OK`: New payload accepted.
  - `202 Accepted`: Duplicate detected (idempotent).
  - `400 Bad Request`: Schema invalid or missing required fields.
  - `401 Unauthorized`: Invalid/revoked/unknown API key.
  - `429 Too Many Requests`: Rate limit hit. Agent backs off next cycle.
  - `5xx`: Server error. Agent retries per backoff algorithm.

3.6 Execution Guardrails (Cron Integration)

3.6.1 Scheduling

3.6.1 Scheduling

```
Bash

# /etc/cron.d/monitoring-agent
* * * * * monitoring-agent flock -n /var/lock/monitoring-
agent.lock /opt/monitoring-agent/venv/bin/python
/opt/monitoring-agent/run.py >> /var/log/monitoring-
agent/cron.log 2>&1
```

- **Lock File:** `flock` prevents overlapping runs if previous cycle hangs.
- **Resource Guardrails:**
  - `nice -n 19` (lowest CPU priority)
  - `ionice -c 3` (idle I/O priority)
  - Hard timeout via `signal.alarm(45)` inside Python

3.6.2 State Management

- No local database or queue. Entirely stateless per execution.
- In-memory buffer only during payload construction.
- Config changes require agent restart or `SIGHUP`.

3.7 Local Logging & Debug Mode

Aspect	Specification
Log Path	`/var/log/monitoring-agent/agent.log`
Format	Structured JSON: `{"ts": "ISO8601", "level": "INFO", "module": "gpu", "msg": "...", "rig_uuid": "uuid"}`
Rotation	`RotatingFileHandler`: 10 MB max, 3 backups
Levels	`DEBUG` (dev only), `INFO`, `WARNING`, `ERROR`
Redaction	API key, password hashes, and full `rig_uuid` masked in logs (`****-****-****-1a2b`)
Debug Mode	`debug_mode: true` in config → logs full payload, disables gzip, increases verbosity, prints stack traces

3.7.1 Log Sanitization Rules

- Never log `api_key`, `Authorization`, or raw `sudo` command output containing sensitive paths.
- `journalctl` errors are truncated to 256 chars per line.
- JSON payload logs are minified; pretty-print only in debug mode.

4. SERVER ARCHITECTURE

This section defines the server-side implementation blueprint, covering Django project structure, authentication/authorization flows, ingestion pipeline mechanics, database schema design (PostgreSQL + TimescaleDB), API contracts, background processing, schema versioning, and audit logging. All components are optimized for a single Ubuntu VPS deployment targeting ~1,000 rigs at 1-minute telemetry intervals.

4.1 Django Project Structure & App Boundaries

The project follows a **domain-driven modular layout** to enforce separation of concerns, simplify testing, and enable independent evolution of ingestion, storage, and UI layers.

4.1.1 Directory Layout

```
gpu_monitor/
├── manage.py
├── config/                    # Django settings, wsgi/asgi, routing
│   ├── settings/              # base.py, dev.py, prod.py
│   ├── urls.py                # Root URL dispatcher
│   └── middleware.py          # Correlation ID, security headers
├── apps/
│   ├── accounts/              # Users, API keys, RBAC, session auth
│   ├── rigs/                  # Rig inventory, tags, ownership, status
│   ├── metrics/               # TimescaleDB ingestion, aggregation, snapshots
│   ├── dashboard/             # HTMX views, polling endpoints, UI templates
│   └── audit/                 # Immutable activity logs, triggers
├── templates/                 # Base + app-specific Django templates
├── static/                    # CSS, JS, HTMX extensions, Chart.js
└── scripts/                   # Management commands, backup helpers
```

4.1.2 App Responsibilities & Dependencies

App	Primary Models	Key Views/Endpoints	Dependencies
`accounts`	`User`, `ApiKey`, `Group`, `Permission`	Login, logout, key management UI, `/auth/` APIs	Django Auth, DRF
`rigs`	`Rig`, `RigTag`, `RigTagging`	`/dashboard/rigs/`, `/dashboard/rigs/<uuid>/`	`accounts`
`metrics`	`MetricSnapshot` (hypertable), `ErrorEvent`	`/api/v1/ingest/`, `/api/v1/rigs/<uuid>/metrics/`	`rigs`, TimescaleDB
`dashboard`	(None - stateless views)	HTMX polling partials, chart data endpoints	`rigs`, `metrics`
`audit`	`AuditLog`	Admin-only query interface	`accounts`, `rigs`

Dependency Rule: `metrics` and `dashboard` must never import `accounts` directly for auth logic. Authentication is handled by DRF middleware and Django's built-in `request.user`.

4.2 Authentication & Authorization Flow

4.2.1 Dual-Context Authentication

Context	Mechanism	Transport	Validation
Agent Ingestion	Custom DRF `APIKeyAuthentication`	`X-API-Key` header	Argon2id hash comparison, `is_active` check, `last_used_at` update
Dashboard UI	Django Session Authentication	Secure, HttpOnly, SameSite=Lax cookie	`SessionMiddleware` + `LoginRequiredMixin`

4.2.2 API Key Validation Middleware

Python

```
# apps/accounts/authentication.py
from rest_framework import authentication, exceptions
from django.contrib.auth.hashers import check_password
from django.utils import timezone
from .models import ApiKey

class APIKeyAuthentication(authentication.BaseAuthentication):
    def authenticate(self, request):
        key = request.headers.get("X-API-Key")

        # 1. Basic validation
        if not key or len(key) < 8:
            return None

        prefix = key[:8]

        try:
            # 2. Secure Lookup: Filter by prefix AND active
status
            api_key_obj =
ApiKey.objects.select_related("user").get(
                prefix=prefix,
                is_active=True
            )
        except ApiKey.DoesNotExist:
            # 3. Timing Attack Mitigation: Perform dummy hash
check if key prefix not found
            check_password(key,
"$argon2id$v=19$m=65536,t=3,p=4$dummy_salt$dummy_hash")
            raise exceptions.AuthenticationFailed("Invalid API
key.")

        # 4. Constant-Time Cryptographic Verification
(Argon2id)
        if not check_password(key, api_key_obj.key_hash):
            raise exceptions.AuthenticationFailed("Invalid API
key.")

        # 5. Update telemetry metadata
        # Note: In a higher-scale system, this would be
batched.
        # For v1 (1000 rigs), a targeted UPDATE on the indexed
PK is acceptable.
        api_key_obj.last_used_at = timezone.now()
        api_key_obj.save(update_fields=["last_used_at"])

        return (api_key_obj.user, api_key_obj)
```

4.2.3 RBAC Enforcement

Role	Permissions	Implementation
Owner	View/edit own rigs, manage own keys, assign tags	`IsOwnerOrReadOnly` permission class
Admin	View all rigs, reassign ownership (audit-logged), manage all users	`IsAdmin` permission class (`user.is_staff=True`)
Viewer	Read-only dashboard access	`IsAuthenticated` + `HasModelLevelRead`

Immutable Ownership Enforcement:

- DB constraint: `UNIQUE(rig_uuid)` + `FOREIGN KEY(owner_id) REFERENCES accounts_user(id)`
- No `PATCH/PUT` endpoint allows `owner_id` mutation.
- Admin reassignment uses a dedicated management command or admin action that creates an `AuditLog` entry and bypasses standard serializer validation.

4.3 Ingestion Pipeline & Request Processing

4.3.1 Request Flow Sequence

Client POST /api/v1/ingest/

→ Nginx (TLS, rate limit, payload size check)

→ DRF APIKeyAuthentication (validate key)

→ DRF Throttle (per-key rate limit)

→ IngestSerializer.validate() (schema version, field mapping, partial failure tolerance)

→ DB UPSERT (UNIQUE constraint + ON CONFLICT DO UPDATE)

→ Response (200/202/4xx/429)

4.3.2 DRF Serializer & Version Routing

```
Python

# apps/metrics/serializers.py
class IngestSerializer(serializers.Serializer):
    schema_version = serializers.CharField()
    rig_uuid = serializers.UUIDField()
    agent_version = serializers.CharField()
    timestamp = serializers.DateTimeField()
```

```
    # Version-specific field mapping handled in
`to_internal_value`
    def to_internal_value(self, data):
        version = data.get("schema_version")
        if version not in SUPPORTED_SCHEMA_VERSIONS:
            raise serializers.ValidationError(f"Unsupported
schema_version: {version}")

        # Map incoming JSON to normalized internal dict
        # Allow missing non-critical fields (partial failure
tolerance)
        # Return normalized payload for DB writer
        return self._normalize_payload(version, data)
```

- **Strict Mode:** Unknown fields are logged and ignored (`extra='allow'` in serializer config).
- **Partial Failures:** If `metrics.gpus` is missing but `inventory` exists, the row is still inserted with `NULL` GPU columns. Server logs `WARNING: Partial payload from {rig_uuid}`.

4.3.3 Idempotency & Conflict Resolution

- **Unique Constraint:** `UNIQUE(rig_uuid, schema_version, timestamp)` on `metrics_metricsnapshot`
- **UPSERT Logic:**

```
Sql

INSERT INTO metrics_metricsnapshot (...)
VALUES (...)
ON CONFLICT (rig_uuid, schema_version, timestamp) DO NOTHING;
```

- **Response Mapping:**
  - `INSERT` → `200 OK`
  - `CONFLICT` → `202 Accepted` (duplicate ignored safely)
  - Server never rejects valid payloads for being duplicates.

4.4 Data Storage Layer (PostgreSQL + TimescaleDB)

4.4.1 Relational Schema (PostgreSQL)

Table	Purpose	Key Columns	Constraints
`accounts_user`	Multi-user auth	`id`, `email`, `password_hash`, `is_staff`, `is_active`	Django default
`accounts_apikey`	Ingestion auth	`id`, `user_id`, `name`, `key_hash`, `is_active`, `created_at`, `last_used_at`	`UNIQUE(name, user_id)`

Table	Purpose	Key Columns	Constraints
rigs_rig	Static inventory + status	uuid (PK), owner_id, name, expected_gpus, last_seen, status	UNIQUE(uuid), FK owner_id
rigs_rigtag	User-defined grouping	id, user_id, name, color	UNIQUE(name, user_id)
rigs_rig_tags	M2M mapping	rig_id, tag_id	Composite PK
audit_auditlog	Immutable activity	id, timestamp, user_id, action, target_type, target_id, metadata_json	CHECK (metadata IS JSON)

4.4.2 Time-Series Schema (TimescaleDB)

```
Sql

-- Hypertable for raw metrics
CREATE TABLE metrics_metricsnapshot (
    time TIMESTAMPTZ NOT NULL,
    rig_uuid UUID NOT NULL,
    schema_version TEXT NOT NULL,
    cpu_util NUMERIC, mem_used BIGINT, gpu_temp NUMERIC,
    gpu_util NUMERIC, gpu_mem_util NUMERIC, disk_usage_pct
NUMERIC,
    payload JSONB, -- Fallback for unparsed/legacy fields
    PRIMARY KEY (time, rig_uuid)
);

-- Convert to hypertable, partition by day
SELECT create_hypertable('metrics_metricsnapshot', 'time',
    chunk_time_interval => INTERVAL '1 day',
    migrate_data => true);

-- Index for dashboard point queries
CREATE INDEX idx_metrics_rig_time ON metrics_metricsnapshot
(rig_uuid, time DESC);
```

4.4.3 Retention & Aggregation Strategy

Layer	Storage	Retention	Mechanism
Raw	`metrics_metricsnapshot`	7 days	`add_drop_chunks_policy('metrics_metricsnapshot', INTERVAL '7 days')`
Hourly Agg	`metrics_hourly_agg` (continuous view)	90 days	Timescale continuous aggregate + refresh policy
Daily Agg	`metrics_daily_agg` (continuous view)	1 year	Timescale continuous aggregate + refresh policy

Continuous Aggregate Definition (Example):

```
Sql

CREATE MATERIALIZED VIEW metrics_hourly_agg
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 hour', time) AS bucket,
    rig_uuid,
    AVG(gpu_temp) AS avg_gpu_temp,
    MAX(gpu_util) AS max_gpu_util,
    AVG(mem_used) AS avg_mem_used
FROM metrics_metricsnapshot
GROUP BY bucket, rig_uuid;

SELECT add_continuous_aggregate_policy('metrics_hourly_agg',
    start_offset => INTERVAL '3 hours',
    end_offset => INTERVAL '1 hour',
    schedule_interval => INTERVAL '1 hour');
```

Query Optimization: Dashboard polls query continuous aggregates, not raw hypertables. Timescale automatically falls back to raw data for the most recent un-refreshed window (`WITH (timescaledb.refresh)`).

4.5 API Layer & DRF Contract

4.5.1 Endpoint Catalog

Method	Path	Purpose	Auth	Rate Limit
POST	/api/v1/ingest/	Telemetry submission	API Key	2/min per key
GET	/api/v1/rigs/	List user rigs (paginated)	Session	10/min
GET	/api/v1/rigs/<uuid>/	Rig detail JSON	Session	20/min
GET	/api/v1/rigs/<uuid>/htmx-metrics/	Dashboard polling partial	Session	20/min
GET	/api/v1/health/	Internal server health	None	1/sec
POST	/accounts/api-keys/	Create/revoke key	Session	5/hr

4.5.2 Response Schema & Error Taxonomy

Code	Meaning	Action
200	Success (new payload)	Agent logs OK
202	Duplicate (idempotent)	Agent logs DUP, ignores
400	Validation failed	Agent logs ERR, retries next cycle
401	Missing/invalid API key	Agent checks config, alerts
403	Forbidden (wrong owner/role)	Server logs audit, rejects
409	UUID already claimed by another user	Server rejects, requires admin
429	Rate limit exceeded	Agent backs off, retries next cycle
5xx	Server error	Agent retries per backoff algorithm

4.5.3 HTMX Polling Contract

- **Endpoint:** `GET /dashboard/rigs/<uuid>/htmx-metrics/`

- **Headers:** `HX-Request: true`, `Cache-Control: no-cache`
- **Response:** `text/html` fragment containing `<div hx-swap-oob="true">` for metric cards
- **Payload:** Lightweight JSON query → continuous aggregates → Django template render
- **Latency Budget:** `< 300ms` (achieved via Timescale index + Django template cache)

4.6 Background Processing, Aggregation & Offline Detection

4.6.1 Design Philosophy (Single VPS Constraint)

- **No Celery/RabbitMQ.** TimescaleDB handles aggregation natively. Django management commands + `systemd timers` handle maintenance tasks.
- **Advantages:** Lower RAM footprint, simpler ops, fewer failure modes, fits 16GB VPS comfortably.

4.6.2 Offline Detection Logic

Runs every 2 minutes via `systemd.timer` → `python manage.py update_rig_status`:

```
Python

# Logic pseudocode
now = timezone.now()
threshold_stale = now - timedelta(minutes=3)
threshold_offline = now - timedelta(minutes=10)

Rig.objects.filter(last_seen__lte=threshold_offline).update(st
atus="offline")
Rig.objects.filter(last_seen__gt=threshold_offline,
last_seen__lte=threshold_stale).update(status="stale")
Rig.objects.filter(last_seen__gt=threshold_stale).update(statu
s="online")
```

- **Dashboard Impact:** Status badge updates automatically via HTMX poll.
- **Configurable:** Thresholds stored in `core_siteconfig` model for future UI tuning.

4.6.3 Cleanup & Maintenance

Task	Mechanism	Frequency	Notes
Raw chunk drop	Timescale policy	Auto (7 days)	`add_drop_chunks_policy`
Continuous aggregate refresh	Timescale policy	Auto (1 hour)	Real-time fallback enabled
Orphaned API key cleanup	Django command	Daily 02:00	Revoked keys > 30d archived
Audit log rotation	DB table partitioning	Monthly	`audit_auditlog` partitioned by month

4.7 Schema Versioning & Migration Strategy

4.7.1 Version Acceptance Matrix

Server Version	Accepts Agent Versions	Behavior
`v1.0`	`schema_version: "1.0"`	Full support
`v1.1` (future)	`"1.0"`, `"1.1"`	`"1.0"` maps to v1.1 model with defaults for new fields. `"1.1"` gets full validation.

4.7.2 Forward/Backward Compatibility Rules

- **Backward:** Server never drops support for `schema_version` until 2 major releases later. Dashboard shows "Agent Upgrade Recommended" if `agent_version` lags.
- **Forward:** Server ignores unknown JSON keys (`extra='allow'` in DRF). Logs `INFO: Ignoring unknown field {key} from rig {uuid}`.
- **Payload Evolution:** New fields are `NULLABLE` on initial deployment. After 30-day rollout, `NOT NULL` constraint added via Django migration.

4.7.3 Agent Upgrade Tracking

- `Rig` model stores `last_agent_version` (updated on each ingest).

- Dashboard admin view filters by `agent_version < "1.1.0"` to target upgrade campaigns.

4.8 Audit Logging & Observability

4.8.1 Immutable Audit Table

```
Sql

CREATE TABLE audit_auditlog (
    id BIGSERIAL PRIMARY KEY,
    timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    user_id UUID REFERENCES accounts_user(id) ON DELETE SET
NULL,
    action VARCHAR(50) NOT NULL, -- e.g., "apikey.create",
"rig.enroll", "key.revoke"
    target_type VARCHAR(50) NOT NULL, -- e.g., "apikey",
"rig", "user"
    target_id UUID,
    ip_address INET,
    metadata JSONB
);
```

- **Triggers:** Django `post_save`/`pre_delete` signals on `ApiKey`, `Rig`, `User`.
- **Immutability:** `DELETE`/`UPDATE` restricted to superuser via DB role. Application code only `INSERT`s.
- **Retention:** Partitioned by month. Retained for 2 years. Archived to cold storage annually.

4.8.2 Structured Logging & Correlation

- **Format:** JSON (`structlog` or Django `logging` with `jsonformatter`)
- **Fields:** `timestamp`, `level`, `logger`, `correlation_id`, `rig_uuid`, `api_key_hash_prefix`, `message`, `duration_ms`
- **Correlation ID Middleware:** Generates UUID per request, propagates to logs and response header `X-Correlation-ID`.
- **Log Rotation:** `logrotate` daily, compress, retain 30 days. Shipped to local disk only (v1 scope).

4.8.3 Health Endpoint (`/api/v1/health/`)

Returns:

```
Json

{
  "status": "healthy",
```

```
  "version": "1.0.0",
  "uptime_seconds": 124800,
  "db_connection": "ok",
  "timescale_version": "2.14.1",
  "last_migration": "0042_add_daily_aggregation",
  "active_rigs": 842,
  "ingest_qps": 14.2
}
```

- **Monitoring:** Uptime Robot / Cronitor polls this endpoint every 60s. Triggers alert if `status != "healthy"` or `db_connection != "ok"`.

5. DASHBOARD SPECIFICATION

This section defines the user interface architecture, interaction models, and performance budgets for the web dashboard. Adhering strictly to the requirements, the UI relies on **Django Templates + HTMX** to deliver a highly interactive, server-rendered experience without the complexity, build-step overhead, or state-management fragility of a Single Page Application (SPA) framework like React or Vue.

5.1 UI Architecture & HTMX Integration Strategy

The dashboard follows a **Hypermedia-Driven Application (HDA)** pattern. The server is the single source of truth for UI state, and HTMX is used exclusively to swap HTML fragments, eliminating full-page reloads and minimizing client-side JavaScript.

5.1.1 Core HTMX Patterns Used

Pattern	HTMX Attributes	Use Case in Dashboard
Polling	`hx-trigger="every 30s"`, `hx-get="..."`	Fleet overview table, rig detail live metrics
Lazy Loading	`hx-trigger="revealed"`, `hx-`	Historical charts, heavy inventory tabs

Pattern	HTMX Attributes	Use Case in Dashboard
	`get="..."`	
Form Submission	`hx-post="..."`, `hx-target="..."`, `hx-swap="outerHTML"`	API key creation, tag assignment, password changes
Out-of-Band (OOB)	`hx-swap-oob="true"`	Updating global toast notifications or nav-bar badge counts during a localized poll
Seamless Updates	`hx-ext="morphdom-swap"`	Prevents scroll-jump and input-focus loss when the fleet table re-renders every 30s

5.1.2 Client-Side JavaScript Constraints

To maintain the "simpler than a full SPA" requirement, custom JS is strictly limited to:

1. **Chart Rendering:** Initializing charting libraries (e.g., Chart.js or uPlot) after HTMX swaps canvas elements.
2. **UI Enhancements:** Lightweight libraries like `Tom Select` for multi-select tag dropdowns.
3. **Event Bridging:** Listening to `htmx:afterSwap` to trigger JS initialization on newly injected DOM nodes. *No client-side routing, no global state stores (Redux/Zustand), no client-side data fetching.*

5.2 Authentication & User Management UI

Built on Django's native `contrib.auth` and `contrib.messages` frameworks, styled with a lightweight CSS framework (e.g., Bootstrap 5 or Tailwind CSS via Django templates).

5.2.1 Auth Flows

- **Login/Logout:** Standard session-based cookie auth. `LoginRequiredMixin` protects all `/dashboard/` routes.
- **Password Change:** Native Django `PasswordChangeView` with HTMX form validation for instant inline error feedback (e.g., "Password too short").

5.2.2 API Key Management Interface

- **Creation:** User clicks "Generate Key". Server creates `ApiKey`, hashes it, and returns an HTMX partial containing the **plaintext key exactly once** inside a copy-to-clipboard UI component.
- **Listing:** Table showing `Name`, `Created`, `Last Used`, `Status`. The hash is never exposed.
- **Revocation:** "Revoke" button triggers a `DELETE` or `POST` with CSRF token. The row is swapped via HTMX to show a "Revoked" badge, and an audit log entry is generated.

5.3 Rig List Page (Fleet Overview)

The fleet overview is the primary landing page. It must render quickly and update seamlessly without disrupting the user's workflow (e.g., losing scroll position or filter state).

5.3.1 Layout & Column Specification

The table is rendered server-side. Columns map directly to the requirements baseline:

Column	Data Source	Rendering Logic / UX
Rig Name	`rigs_rig.name`	Clickable link to Detail Page.
Status	`rigs_rig.status`	Badge: □ Online, □ Stale, □ Offline (based on Sec 4.6.2 thresholds).
Last Seen	`rigs_rig.last_seen`	Relative time (e.g., "2m ago", "1h ago") via Django `naturaltime`.

Column	Data Source	Rendering Logic / UX
Health Summary	Derived	□ OK, ⚠□ Warning (high temp/stale), □ Critical (offline/errors).
Latest Error	`metrics_error`	Red `!` icon with tooltip showing the latest kernel/NVIDIA/Docker error.
Tags	`rigs_rigtag`	Colored pills (e.g., `[prod] [inference]`).
GPU Util	Latest snapshot	Progress bar (0-100%).
GPU Mem Util	Latest snapshot	Progress bar + text (e.g., "22/24 GB").
CPU Temp	Latest snapshot	Text with color coding (e.g., >85°C = red).
Disk Usage	Latest snapshot	Max usage across all mounted partitions.

5.3.2 HTMX Polling & Morphing Implementation

To achieve the 30-second refresh requirement without UI flicker:

```
Html

<!-- Fleet Table Container -->
<table class="table">
  <thead>...</thead>
  <tbody id="fleet-body"
         hx-get="{% url 'dashboard:fleet_partial' %}"
         hx-trigger="every 30s"
         hx-swap="innerHTML"
         hx-ext="morphdom-swap"
         hx-include="#filter-form">
    <!-- Initial rows rendered by Django -->
    {% include "dashboard/partials/fleet_rows.html" %}
  </tbody>
```

```
</table>
```

- `hx-include`: Ensures current search/filter parameters are passed to the polling endpoint.
- `morphdom-swap`: The HTMX morphdom extension diffs the DOM, updating only changed text nodes (like "Last Seen" or "GPU Util") without destroying and recreating the `<tr>` elements. This preserves scroll position and hover states.

5.3.3 Filtering & Search

- **Mechanism:** URL query parameters (`?status=online&tag=prod&search=node-01`).
- **UI:** A filter bar with HTMX `hx-get` triggers on input/change.
- **Backend:** Django ORM `Q` objects filter the `Rig` queryset. Admin users bypass the `owner=request.user` filter.

5.4 Rig Detail Page

The detail page is divided into three logical sections, implemented as tabs to keep the initial page load lightweight.

5.4.1 Tab 1: Static Inventory

Rendered immediately on page load.

- **Hardware Grid:** Cards for CPU, Motherboard, Memory, Storage (table of disks + SMART health), Network (interfaces + IPs), and GPU Inventory (table of all detected GPUs with UUIDs and PCIe specs).
- **Software Grid:** OS, Kernel, Uptime, NTP status, NVIDIA Driver, CUDA, Docker version.
- **Data Source:** The most recent `inventory` JSONB payload from the `metrics_metricsnapshot` table, parsed and formatted by Django template filters.

5.4.2 Tab 2: Live Status & Errors

Updates via 30s HTMX poll.

- **Current Vitals:** Large typography for current CPU/GPU temps, load averages, and power draw.
- **Docker State:** Table of running containers, image tags, uptime, and restart counts. Highlight containers with `restart_count > 0` in yellow.
- **Latest Errors Panel:**
  - Dedicated section for the latest aggregated errors (Kernel, NVIDIA, Docker/Systemd).
  - Displays: `Timestamp`, `Source`, `Message`.
  - *Constraint Check:* Meets the requirement to "store latest errors with date... do not ingest full log history".

5.4.3 Tab 3: Historical Charts

Lazy-loaded via `hx-trigger="revealed"` to prevent blocking the initial page render.

- **Charting Library: Chart.js** (with `chartjs-adapter-date-fns` for time axes) or **uPlot** (for higher performance with dense time-series data).
- **Time Range Selector:** Dropdown (1H, 6H, 24H, 7D, 30D) that triggers an HTMX GET to fetch new JSON data.
- **Required Graphs (Mapped to KB Sec 5.5):**

1. GPU Temperature (Line, multi-series if >1 GPU)
2. GPU Utilization (Line, %)
3. GPU Memory Utilization (Line, %)
4. GPU Power Draw (Line, Watts)
5. CPU Load (Line, 1m/5m/15m averages)
6. CPU Temperature (Line)
7. RAM Usage (Stacked Area: Used/Cached/Free)
8. Disk Usage (Line, % full for primary mounts)

Chart Data API Contract

The frontend JS requests chart data from a dedicated JSON endpoint: `GET /dashboard/rigs/<uuid>/chart-data/?metric=gpu_temp&range=24h`

Backend Optimization (TimescaleDB): To ensure charts load in `< 500ms`, the backend queries the **Continuous Aggregates** (defined in Sec 4.4.3) rather than raw minute-level data for ranges > 24H.

```
Python

# Pseudocode for Chart Data View
if range == "24h":
    # Query raw hypertable, downsample to 1-minute buckets
    data = MetricSnapshot.objects.filter(...).time_bucket('1
minute')
elif range == "7d":
    # Query hourly continuous aggregate
    data = HourlyAgg.objects.filter(...).time_bucket('1 hour')
elif range == "30d":
    # Query daily continuous aggregate
    data = DailyAgg.objects.filter(...).time_bucket('1 day')
```

5.5 Tagging & Filtering System

Tags provide operational grouping (e.g., `production`, `lab`, `customer-a`).

5.5.1 Data Model & UI

- **Model:** `RigTag` (name, color, owner) and `RigTagging` (M2M through table).
- **Assignment UI:** A multi-select dropdown (using Tom Select) on the Rig Detail page. Selecting a tag fires an HTMX POST to `/dashboard/rigs/<uuid>/tags/`.
- **Color Coding:** Tags have predefined or user-selectable hex colors. The Rig List page renders these as colored pills.
- **Filtering:** The Rig List page includes a "Filter by Tag" dropdown. Selecting a tag updates the URL query string and triggers an HTMX swap of the fleet table.

5.6 Performance, Caching & UX Budgets

To ensure the dashboard feels instantaneous despite polling and heavy time-series queries, the following budgets and strategies are enforced:

Metric	Target Budget	Implementation Strategy
Initial Page Load (List)	`< 800ms`	`select_related('owner')`, `prefetch_related('tags')`. No N+1 queries.
30s Polling Payload	`< 50 KB`	Return only `<tr>` fragments. Minify HTML. Exclude unchanged static data.
Chart Data Fetch	`< 500ms`	Query TimescaleDB Continuous Aggregates. Limit JSON payload to `< 2000` data points per chart via backend downsampling.
Template Rendering	`< 100ms`	Use Django `{% cache %}` template tags for static sidebar elements and complex tag pills.

Metric	Target Budget	Implementation Strategy
UX: Scroll Preservation	100% success	Use `morphdom` HTMX extension for table swaps.
UX: Empty States	N/A	Graceful UI for new users (e.g., "No rigs found. Run this script to enroll your first rig...").

5.6.1 Error Handling in UI

- **Agent Offline:** If a rig is offline, the Live Status tab shows a prominent banner: *"Rig has not reported in X hours. Last seen: [Timestamp]."* Charts gracefully degrade, showing data only up to the last known timestamp without breaking the time axis.
- **Partial Data:** If GPU metrics are missing (e.g., agent running on a CPU-only node), the GPU charts are hidden via template logic (`{% if rig.has_gpus %}`), preventing empty/broken chart renders.

6. DATA MODEL & SCHEMA VERSIONING

This section provides the exact implementation blueprint for the database schema, JSON contracts, and versioning protocols. The data model is strictly partitioned into the **five logical layers** mandated by the requirements baseline to prevent mixing transient time-series data with static inventory or audit trails.

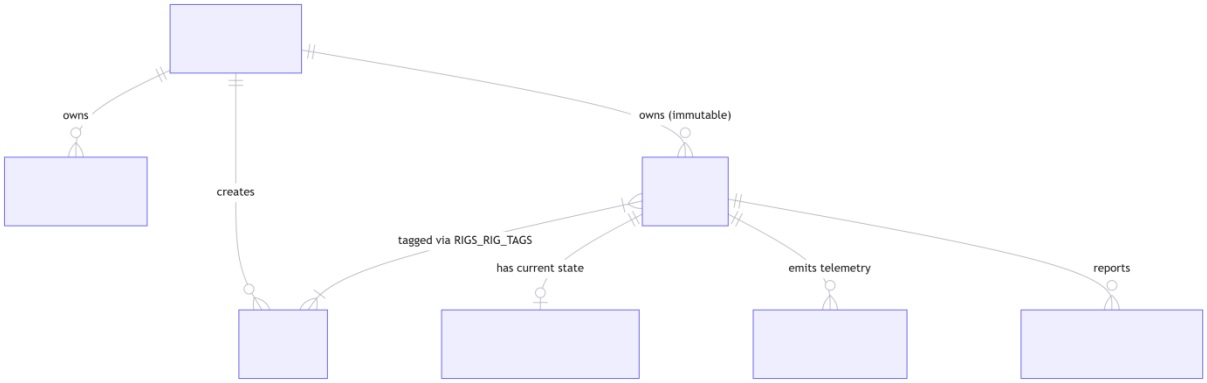
6.1 Layer 1: Relational Schema (Static Inventory & Identity)

This layer manages user identity, access control, rig ownership, and operational grouping. It uses standard PostgreSQL tables managed by Django ORM migrations.

6.1.1 Entity-Relationship Overview

```
Mermaid
```

```
erDiagram
    ACCOUNTS_USER ||--o{ ACCOUNTS_APIKEY : "owns"
    ACCOUNTS_USER ||--o{ RIGS_RIG : "owns (immutable)"
    ACCOUNTS_USER ||--o{ RIGS_TAG : "creates"
    RIGS_RIG }|--|{ RIGS_TAG : "tagged via RIGS_RIG_TAGS"
    RIGS_RIG ||--o| METRICS_LATEST_SNAPSHOT : "has current
state"
    RIGS_RIG ||--o{ METRICS_TIMESERIES : "emits telemetry"
    RIGS_RIG ||--o{ METRICS_LATEST_ERRORS : "reports"
```



6.1.2 Core Table DDL (PostgreSQL)

`accounts_apikey` (Secures agent ingestion)

```
sql

CREATE TABLE accounts_apikey (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES accounts_user(id) ON
DELETE CASCADE,
    name VARCHAR(64) NOT NULL,
    prefix VARCHAR(8) NOT NULL UNIQUE,      -- For UI display
(e.g., "mk-8f2a")
    key_hash VARCHAR(255) NOT NULL,         -- Argon2id hash
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    last_used_at TIMESTAMPTZ NULL,
    CONSTRAINT unique_name_per_user UNIQUE (user_id, name)
);
```

```
CREATE INDEX idx_apikey_prefix ON accounts_apikey(prefix);
```

`rigs_rig` (The immutable asset registry)

```
sql

CREATE TABLE rigs_rig (
    uuid UUID PRIMARY KEY,                  -- Matches agent's
generated UUID
    owner_id UUID NOT NULL REFERENCES accounts_user(id) ON
DELETE RESTRICT,
    name VARCHAR(128) NOT NULL DEFAULT 'Unnamed Rig',
    expected_gpu_count INTEGER DEFAULT 0,
    status VARCHAR(16) DEFAULT 'offline',   -- online, stale,
offline
    last_seen TIMESTAMPTZ NULL,
    last_agent_version VARCHAR(32) NULL,
    enrolled_at TIMESTAMPTZ DEFAULT NOW(),
    CONSTRAINT chk_status CHECK (status IN ('online', 'stale',
'offline'))
);
CREATE INDEX idx_rig_owner ON rigs_rig(owner_id);
CREATE INDEX idx_rig_status ON rigs_rig(status);
```

`rigs_tag` & `rigs_rig_tags` (Fleet grouping)

```
sql

CREATE TABLE rigs_tag (
    id SERIAL PRIMARY KEY,
    owner_id UUID NOT NULL REFERENCES accounts_user(id) ON
DELETE CASCADE,
    name VARCHAR(64) NOT NULL,
    color_hex VARCHAR(7) DEFAULT '#6c757d',
    CONSTRAINT unique_tag_name_per_user UNIQUE (owner_id,
name)
);

CREATE TABLE rigs_rig_tags (
    rig_uuid UUID REFERENCES rigs_rig(uuid) ON DELETE CASCADE,
    tag_id INTEGER REFERENCES rigs_tag(id) ON DELETE CASCADE,
    PRIMARY KEY (rig_uuid, tag_id)
```

```
);
```

6.2 Layer 2: Latest Snapshot (Current State)

To ensure the dashboard loads instantly without querying time-series aggregates for the "current" state, we maintain a single row per rig containing the most recent payload.

`metrics_latest_snapshot`

```
sql

CREATE TABLE metrics_latest_snapshot (
    rig_uuid UUID PRIMARY KEY REFERENCES rigs_rig(uuid) ON
DELETE CASCADE,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    -- Extracted high-value metrics for fast UI rendering
    cpu_util NUMERIC(5,2),
    cpu_temp NUMERIC(5,2),
    mem_used_bytes BIGINT,
    mem_total_bytes BIGINT,
    gpu_util_max NUMERIC(5,2),
    gpu_mem_used_max BIGINT,
    gpu_temp_max NUMERIC(5,2),

    -- Full nested payload for the "Static Inventory" & "Live
Status" tabs
    inventory_json JSONB NOT NULL,
    software_json JSONB NOT NULL,
    docker_json JSONB NOT NULL,
    ai_processes_json JSONB NOT NULL
);
```

Implementation Note: The ingestion pipeline performs an `INSERT ... ON CONFLICT (rig_uuid) DO UPDATE` on this table for every valid 1-minute payload.

6.3 Layer 3: Time-Series Metric History (TimescaleDB)

This layer stores the numerical data required for historical charts (Sec 5.4.3). To optimize write throughput and query speed, we use a flat, wide-table design rather than an Entity-Attribute-Value (EAV) model, leveraging TimescaleDB's native compression.

`metrics_timeseries` (Hypertable)

```
sql

CREATE TABLE metrics_timeseries (
    time TIMESTAMPTZ NOT NULL,
```

```
    rig_uuid UUID NOT NULL REFERENCES rigs_rig(uuid) ON DELETE
CASCADE,

    -- CPU & Memory
    cpu_load_1m NUMERIC(6,2),
    cpu_util NUMERIC(5,2),
    cpu_temp NUMERIC(5,2),
    mem_used_bytes BIGINT,

    -- GPU Metrics (Arrays to support multi-GPU rigs natively)
    gpu_utils NUMERIC(5,2)[],
    gpu_temps NUMERIC(5,2)[],
    gpu_mem_utils NUMERIC(5,2)[],
    gpu_power_draws NUMERIC(5,2)[],

    -- Storage & Network
    disk_usage_max NUMERIC(5,2),
    net_rx_bytes_rate BIGINT,
    net_tx_bytes_rate BIGINT
);

-- Convert to Hypertable (Partitioned by 1 day)
SELECT create_hypertable('metrics_timeseries', 'time',
    chunk_time_interval => INTERVAL '1 day');

-- Index for fast single-rig chart queries
CREATE INDEX idx_timeseries_rig_time ON metrics_timeseries
(rig_uuid, time DESC);

-- Compression Policy (Compress chunks older than 7 days)
ALTER TABLE metrics_timeseries SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'rig_uuid',
    timescaledb.compress_orderby = 'time DESC'
);
SELECT add_compression_policy('metrics_timeseries', INTERVAL
'7 days');
```

6.3.1 Continuous Aggregates (Retention Strategy)

To satisfy the requirement for hourly and daily long-term retention without querying raw minute-level data:

```
Sql

-- Hourly Aggregation (Retained for 90 days)
CREATE MATERIALIZED VIEW metrics_hourly_agg
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 hour', time) AS bucket,
    rig_uuid,
```

```
    AVG(cpu_util) AS avg_cpu_util,
    MAX(cpu_temp) AS max_cpu_temp,
    AVG(gpu_utils[1]) AS avg_gpu_util_0, -- Example for GPU 0
    MAX(gpu_temps[1]) AS max_gpu_temp_0
FROM metrics_timeseries
GROUP BY bucket, rig_uuid;

SELECT add_continuous_aggregate_policy('metrics_hourly_agg',
    start_offset => INTERVAL '3 hours',
    end_offset => INTERVAL '1 hour',
    schedule_interval => INTERVAL '1 hour');

SELECT add_retention_policy('metrics_hourly_agg', INTERVAL '90
days');

-- Daily Aggregation (Retained for 1 year)
CREATE MATERIALIZED VIEW metrics_daily_agg ... -- (Similar
structure, 1 day bucket)
SELECT add_retention_policy('metrics_daily_agg', INTERVAL '1
year');

-- Raw Data Retention (Drop minute-level data after 14 days)
SELECT add_retention_policy('metrics_timeseries', INTERVAL '14
days');
```

6.4 Layer 4: Latest Errors

The requirements explicitly state: *"Only the latest errors with date should be stored for now... do not ingest full log history."*

`metrics_latest_errors`

```
sql

CREATE TABLE metrics_latest_errors (
    id BIGSERIAL PRIMARY KEY,
    rig_uuid UUID NOT NULL REFERENCES rigs_rig(uuid) ON DELETE
CASCADE,
    source VARCHAR(32) NOT NULL, -- 'kernel', 'nvidia',
'docker', 'systemd'
    message TEXT NOT NULL,
    first_seen TIMESTAMPTZ NOT NULL,
```

```
    last_seen TIMESTAMPTZ NOT NULL,
    occurrence_count INTEGER DEFAULT 1,
    CONSTRAINT unique_rig_source UNIQUE (rig_uuid, source,
message)
);
CREATE INDEX idx_errors_rig ON metrics_latest_errors(rig_uuid,
last_seen DESC);
```

Ingestion Logic: The agent sends a deduplicated list of recent errors. The server uses `INSERT ... ON CONFLICT (rig_uuid, source, message) DO UPDATE SET last_seen = EXCLUDED.last_seen, occurrence_count = metrics_latest_errors.occurrence_count + 1`. A periodic background task prunes errors where `last_seen < NOW() - INTERVAL '7 days'`.

6.5 Layer 5: Audit & Activity Events

An immutable ledger for security and compliance (Sec 4.8).

`audit_event`

```
sql

CREATE TABLE audit_event (
    id BIGSERIAL PRIMARY KEY,
    timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    actor_id UUID REFERENCES accounts_user(id) ON DELETE SET
NULL,
    action VARCHAR(64) NOT NULL,       -- e.g.,
'rig.enrolled', 'apikey.revoked'
    target_type VARCHAR(32) NOT NULL,  -- e.g., 'rig',
'apikey'
    target_id UUID NOT NULL,
```

```
    ip_address INET,
    metadata JSONB,
    CONSTRAINT chk_action CHECK (action IN (
        'user.login', 'user.password_change',
        'apikey.created', 'apikey.revoked',
        'rig.enrolled', 'rig.reassigned_admin',
        'tag.created', 'tag.deleted'
    ))
) PARTITION BY RANGE (timestamp);
```

Implementation Note: Table is partitioned by month. Application code only has `INSERT` privileges. `DELETE`/`UPDATE` are restricted to the database superuser.

6.6 Agent Payload JSON Schema (v1.0 Contract)

This is the strict contract the Python Agent (Sec 3) must fulfill. The server validates this using `jsonschema` or DRF serializers before writing to the DB.

```
Json

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Rig Monitor Ingest Payload v1.0",
  "type": "object",
  "required": ["rig_uuid", "schema_version", "agent_version",
"timestamp", "inventory", "metrics"],
  "properties": {
    "rig_uuid": { "type": "string", "format": "uuid" },
    "schema_version": { "type": "string", "enum": ["1.0"] },
    "agent_version": { "type": "string", "pattern":
"^\\d+\\.\\d+\\.\\d+$" },
    "timestamp": { "type": "string", "format": "date-time" },
    "inventory": {
      "type": "object",
      "properties": {
        "cpu": { "type": "object", "properties": { "model":
{"type":"string"}, "cores_physical": {"type":"integer"} } },
        "gpus": {
          "type": "array",
          "items": {
            "type": "object",
            "required": ["uuid", "model", "mem_total_mb"],
            "properties": {
              "uuid": { "type": "string" },
              "pci_bus_id": { "type": "string" },
              "model": { "type": "string" },
              "mem_total_mb": { "type": "integer" }
            }
          }
```

```
        }
      }
    },
    "metrics": {
      "type": "object",
      "required": ["cpu_util", "mem_used_bytes"],
      "properties": {
        "cpu_util": { "type": "number", "minimum": 0,
"maximum": 100 },
        "cpu_temp": { "type": ["number", "null"] },
        "mem_used_bytes": { "type": "integer" },
        "gpu_utils": { "type": "array", "items": { "type":
"number" } },
        "gpu_temps": { "type": "array", "items": { "type":
["number", "null"] } },
        "errors": {
          "type": "array",
          "items": {
            "type": "object",
            "required": ["source", "message", "timestamp"],
            "properties": {
              "source": { "type": "string", "enum": ["kernel",
"nvidia", "docker", "systemd"] },
              "message": { "type": "string", "maxLength": 1024
},
              "timestamp": { "type": "string", "format":
"date-time" }
            }
          }
        }
      }
    }
  }
}
```

6.7 Schema Versioning Protocol

To satisfy the requirement that *"old clients should not break immediately when new fields are introduced"*, the system enforces a strict **Additive-Only Evolution Protocol** for `schema_version`.

6.7.1 Versioning Rules

1. **Minor Changes (Additive):** Adding a new optional field (e.g., `gpu_pcie_gen`) does **not** bump the `schema_version`. The server's JSON schema ignores unknown fields (`additionalProperties: true` in validation logic). The agent continues sending `schema_version: "1.0"`.
2. **Major Changes (Breaking):** Renaming a field, changing a data type (e.g., `cpu_temp` from Number to String), or making an optional field strictly required **must** bump the version to `"2.0"`.
3. **Server Multi-Tenancy:** The server's ingestion router must support multiple serializers simultaneously.

6.7.2 Server-Side Routing Logic

```
Python

# apps/metrics/ingest_router.py
SERIALIZER_MAP = {
    "1.0": IngestSerializerV1,
    "1.1": IngestSerializerV1_1, # Handles new AI
fragmentation metrics
}

def route_payload(payload):
    version = payload.get("schema_version")
    if version not in SERIALIZER_MAP:
        raise UnsupportedSchemaVersionError(version)

    serializer = SERIALIZER_MAP[version](data=payload)
    serializer.is_valid(raise_exception=True)

    # Normalize to internal canonical format before DB write
    return serializer.to_canonical_internal_format()
```

6.7.3 Agent Deprecation Lifecycle

1. **Day 0:** Server v1.2 deployed. Accepts `1.0` and `1.1`.
2. **Day 30:** Dashboard UI flags rigs running `< 1.1` with an "Upgrade Recommended" badge.
3. **Day 180:** Server v2.0 deployed. `schema_version: "1.0"` is officially deprecated but still accepted via a legacy shim that maps old fields to new DB columns.
4. **Day 365:** Support for `"1.0"` is dropped. Agents sending `"1.0"` receive `400 Bad Request`.

6.8 Data Migration & Rollout Strategy

Because the system relies on a single VPS and a monolithic Django app, schema migrations must be executed with zero downtime.

6.8.1 Relational Migrations (Django)

- All changes to Layer 1 and Layer 5 tables are handled via standard `python manage.py makemigrations` and `migrate`.
- **Rule:** No `RenameField` or `DeleteModel` in a single deployment.
  - *Step 1:* Add new field, deploy.
  - *Step 2:* Update code to write to both old and new fields, deploy.
  - *Step 3:* Backfill data via management command.
  - *Step 4:* Update code to read only from new field, deploy.
  - *Step 5:* Drop old field, deploy.

6.8.2 Time-Series Migrations (TimescaleDB)

- Adding a new metric column to `metrics_timeseries` is done via `ALTER TABLE ... ADD COLUMN`. TimescaleDB handles this gracefully across existing chunks.
- **Rule:** Never change the data type of an existing time-series column. If a type change is needed, add a new column (e.g., `gpu_temp_c` -> `gpu_temp_f`), update the agent payload schema, and deprecate the old column.

7. SECURITY ARCHITECTURE

This section translates the security and compliance requirements (KB Sec 10) into concrete, enforceable mechanisms across the network, application, and data layers. The architecture relies on **defense-in-depth**, assuming that the external fleet (Z1) is inherently hostile and that the single VPS environment must be hardened against both external attacks and internal misconfigurations.

7.1 Threat Model (STRIDE Analysis)

The following STRIDE analysis is tailored specifically to the telemetry ingestion and multi-tenant dashboard context.

Threat Category	Specific Attack Vector	Mitigation Strategy	Enforcement Layer
Spoofing	Attacker sends fake metrics for a victim's `rig_uuid` to hide downtime or trigger false alarms.	**Strict Ownership Binding:** If `rig_uuid` exists in DB, the `X-API-Key` must belong to the rig's `owner_id`. First-seen binds UUID to key owner.	DRF Permission Class (`IsRigOwner`)
Tampering	MITM alters payload in transit	**Transport Encryption:** Mandatory	Nginx / Let's Encrypt

Threat Category	Specific Attack Vector	Mitigation Strategy	Enforcement Layer
	(e.g., masking GPU overheating).	TLS 1.3. HSTS enforced. (Payload signing deferred to v2 to maintain agent simplicity).	
Repudiation	Admin reassigns rig ownership or revokes a key and denies the action.	**Immutable Audit Ledger:** All state-changing actions write to the append-only `audit_event` table with actor IP and timestamp.	Django Service Layer / DB Triggers
Information Disclosure	Database backup is stolen; attacker extracts plaintext API keys to impersonate agents.	**Cryptographic Hashing:** API keys are never stored in plaintext. Argon2id hashing ensures stolen hashes are computationally infeasible to reverse.	Django `make_password()`
Denial of Service	Attacker floods `/api/v1/ingest/` with massive payloads or high-frequency requests, exhausting VPS CPU/RAM.	**Multi-Layer Throttling:** Nginx drops excessive IPs; DRF throttles specific API keys; Nginx rejects payloads > 2MB.	Nginx + DRF Throttles
Elevation of Privilege	User A manipulates URL/IDOR to view User B's rig details or delete User B's API keys.	**QuerySet Filtering:** All DRF ViewSets and Django Views strictly filter by `request.user` (unless `is_staff=True`).	Django ORM / DRF `get_queryset()`

7.2 API Key Lifecycle & Cryptography

API keys are the sole authentication mechanism for the client agents. Their lifecycle is strictly managed to ensure compromise containment.

7.2.1 Key Generation & Storage

- **Generation:** Server generates a 48-byte URL-safe string using Python's `secrets.token_urlsafe(48)`.
- **Prefix Extraction:** The first 8 characters are stored in plaintext (`prefix` column) to allow users to identify keys in the UI (e.g., `gr-8f2a...`).
- **Hashing:** The full key is hashed using **Argon2id** before storage.

```
Python
```

```
# settings.py
PASSWORD_HASHERS = [
    "django.contrib.auth.hashers.Argon2PasswordHasher", # Requires
argon2-cffi
    "django.contrib.auth.hashers.PBKDF2PasswordHasher",
]
```

- **Storage:** Only `prefix`, `key_hash`, `user_id`, and metadata are saved to `accounts_apikey`. The plaintext key is returned to the user **exactly once** via the UI and immediately discarded from server memory.

7.2.2 Validation & Constant-Time Comparison

To prevent timing attacks (where an attacker guesses a key character-by-character based on response latency), validation uses Django's built-in `check_password()`, which enforces constant-time comparison.

```
Python

# apps/accounts/authentication.py
from django.contrib.auth.hashers import check_password

def validate_api_key(plaintext_key: str) -> Optional[ApiKey]:
    prefix = plaintext_key[:8]
    try:
        key_obj = ApiKey.objects.get(prefix=prefix,
is_active=True)
    except ApiKey.DoesNotExist:
        # Perform dummy hash check to prevent timing
discrepancies
        check_password(plaintext_key, "dummy_hash")
        return None

    if check_password(plaintext_key, key_obj.key_hash):
        return key_obj
    return None
```

7.2.3 Revocation & Rotation

- **Revocation:** Setting `is_active = False` takes effect on the very next agent request (no caching of auth states).
- **Rotation:** Users generate a new key, update the agent's `config.yaml`, and revoke the old key. Because each rig uses a single key, rotating a key requires updating the specific agents using it.

7.3 Transport Security

All network communication into the VPS is encrypted and strictly controlled.

7.3.1 Nginx TLS Configuration

- **Protocol:** TLS 1.3 only (TLS 1.2 disabled to prevent downgrade attacks).
- **Cipher Suites:** Modern, AEAD ciphers only (e.g., `TLS_AES_256_GCM_SHA384`).
- **HSTS:** `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`
- **Certificate Management:** Let's Encrypt via `certbot`. Auto-renewal handled by a systemd timer (`certbot renew --quiet --deploy-hook "systemctl reload nginx"`).

7.3.2 Nginx Security Headers

```
Nginx

add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin"
always;
add_header Content-Security-Policy "default-src 'self';
script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-
inline'; img-src 'self' data:; font-src 'self';" always;
```

Note on CSP: `'unsafe-inline'` is permitted for scripts/styles temporarily to support HTMX and Django template injections, but will be tightened using nonces in future iterations.

7.4 Rate Limiting & Abuse Prevention

To protect the single VPS from resource exhaustion, rate limiting is applied at both the edge (Nginx) and the application (DRF).

7.4.1 Edge Layer (Nginx)

Protects against network-level floods and large payload attacks.

```
Nginx

# /etc/nginx/conf.d/rate_limiting.conf
# Limit by IP: 30 requests per second (burst 50)
limit_req_zone $binary_remote_addr zone=ip_flood:10m
rate=30r/s;
```

```
# Restrict payload size to prevent RAM exhaustion during JSON
parsing
client_max_body_size 2M;

server {
    # ...
    location /api/v1/ingest/ {
        limit_req zone=ip_flood burst=50 nodelay;
        proxy_pass http://127.0.0.1:8000;
    }
}
```

7.4.2 Application Layer (DRF)

Protects against compromised API keys spamming the database.

```
Python

# settings.py
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.ScopedRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'ingest': '2/min',    # Agents report every 60s. 2/min
allows 1 retry.
        'dashboard': '60/min' # UI polling every 30s per rig.
    }
}

# apps/metrics/views.py
class IngestView(APIView):
    throttle_scope = 'ingest'
    # ...
```

7.5 Audit Logging & Immutability

The `audit_event` table (defined in Sec 6.5) is the system's source of truth for compliance and forensic analysis.

7.5.1 Audited Events

Action	Trigger	Metadata Captured
`apikey.created`	User generates key via UI	Key prefix, IP address
`apikey.revoked`	User/Admin disables key	Key prefix, IP address

Action	Trigger	Metadata Captured
`rig.enrolled`	First valid payload received	`rig_uuid`, Agent version, IP address
`rig.reassigned`	Admin manually changes `owner_id`	Old owner ID, New owner ID, Admin ID
`tag.created` / `tag.deleted`	User modifies fleet grouping	Tag name, Target rig UUIDs
`user.login_failed`	Django auth failure	Username attempted, IP address

7.5.2 Implementation Mechanism

Audit logs are written via a dedicated service class, **not** via Django signals, to ensure explicit control over what is logged and to capture request context (like IP addresses) reliably.

```
Python

# apps/audit/services.py
def log_audit_event(request, action: str, target_type: str,
target_id: str, metadata: dict = None):
    AuditEvent.objects.create(
        actor=request.user if request.user.is_authenticated
else None,
        action=action,
        target_type=target_type,
        target_id=target_id,
        ip_address=get_client_ip(request),
        metadata=metadata or {}
    )
```

7.5.3 Immutability & Retention

- **DB Level:** The `monitoring` app user has `INSERT` and `SELECT` privileges on `audit_event`, but **no** `UPDATE` or `DELETE` privileges.
- **Retention:** Table is partitioned by month. A `systemd` timer runs a management command on the 1st of every month to detach partitions older than 24 months, compress them, and move them to cold storage (e.g., S3 or local archive).

7.6 Secret Storage & Configuration Security

To prevent credential leakage, the application strictly separates code from configuration.

7.6.1 Environment Variables

- All secrets (`DJANGO_SECRET_KEY`, `DB_PASSWORD`, `REDIS_URL` if added later) are injected via environment variables.
- **No hardcoded secrets** in `settings.py`.
- Managed via `python-dotenv` reading from a `.env` file in production, or natively via systemd `EnvironmentFile`.

7.6.2 File System Permissions

```
Bash

# .env file permissions
chown monitoring:monitoring /opt/monitoring-app/.env
chmod 0600 /opt/monitoring-app/.env

# Django settings directory
chmod -R 0755 /opt/monitoring-app/config/
```

7.6.3 Database Credentials

- The Django app connects to PostgreSQL using a dedicated role (`app_user`) with a strong, randomly generated password.
- `pg_hba.conf` restricts connections to `127.0.0.1/32` using `scram-sha-256` authentication.
- The `postgres` superuser password is set but kept offline in a secure password manager; it is only used for initial setup and major maintenance.

8. OPERATIONAL ARCHITECTURE

This section defines the day-two operations blueprint for the single Ubuntu VPS deployment. It translates the operational requirements (KB Sec 11) into concrete runbooks, systemd configurations, backup strategies, and upgrade procedures. The guiding principle is **operational simplicity**: leveraging native Linux tooling (systemd, cron, journald) to minimize moving parts while ensuring high availability and data durability for the ~1,000 rig fleet.

8.1 Deployment Runbook & Process Management

The application is deployed as a standard Linux service stack. Infrastructure as Code (e.g., Ansible/Terraform) is recommended for initial provisioning, but day-to-day operations rely on standard package management and systemd.

8.1.1 Directory Structure & Permissions

```
Text

/opt/monitoring-app/           # Django project root (owned by
monitoring:monitoring)
├── venv/                      # Python virtual environment
├── config/                    # Django settings
├── manage.py
├── static/                    # Collected static assets (Nginx
reads this)
```

```
└── .env                          # Environment variables (0600
permissions)

/var/log/monitoring/              # Application logs (owned by
monitoring:adm)
/etc/nginx/sites-available/       # Nginx virtual hosts
/etc/systemd/system/              # Systemd unit files
```

8.1.2 Systemd Unit: Gunicorn Application

Ensures the Django app starts on boot, restarts on failure, and supports zero-downtime reloads.

```
Ini

# /etc/systemd/system/gunicorn.service
[Unit]
Description=gunicorn daemon for GPU Monitor
Requires=gunicorn.socket
After=network.target postgresql.service

[Service]
Type=notify
User=monitoring
Group=monitoring
RuntimeDirectory=gunicorn
WorkingDirectory=/opt/monitoring-app
Environment="DJANGO_SETTINGS_MODULE=config.settings.prod"
EnvironmentFile=/opt/monitoring-app/.env
ExecStart=/opt/monitoring-app/venv/bin/gunicorn --access-
logfile - --workers 4 --bind unix:/run/gunicorn/gunicorn.sock
config.wsgi:application
ExecReload=/bin/kill -s HUP $MAINPID
KillMode=mixed
TimeoutStopSec=5
PrivateTmp=true
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Zero-Downtime Deploy: The `ExecReload` directive sends a `SIGHUP` to the master process, which gracefully spawns new workers and bleeds off old workers without dropping active connections.

8.1.3 Background Task Scheduling (Cron)

Instead of heavy task queues (Celery), maintenance tasks run via the `monitoring` user's crontab.

```
Bash
```

```
# crontab -u monitoring
# Offline/Stale detection (runs every 2 minutes)
*/2 * * * * cd /opt/monitoring-app && ./venv/bin/python
manage.py update_rig_status >> /var/log/monitoring/cron.log
2>&1

# Prune old error records (runs daily at 01:00)
0 1 * * * cd /opt/monitoring-app && ./venv/bin/python
manage.py prune_latest_errors >> /var/log/monitoring/cron.log
2>&1

# Database backup (runs daily at 02:00)
0 2 * * * /opt/monitoring-app/scripts/backup_db.sh >>
/var/log/monitoring/backup.log 2>&1
```

8.2 Backup & Disaster Recovery

Data loss for telemetry and inventory is unacceptable. The backup strategy uses native PostgreSQL tools combined with offsite storage.

8.2.1 Backup Strategy

Component	Tool	Frequency	Retention Policy	Storage Location
Relational DB (Inventory, Users, Audit)	`pg_dump` (Custom format)	Daily @ 02:00	7 Daily, 4 Weekly, 6 Monthly	Offsite S3/B2 via `rclone`
Time-Series DB (TimescaleDB)	`pg_dump` + Continuous Aggregates	Daily @ 02:00	Same as above	Offsite S3/B2 via `rclone`
Configuration (.env, Nginx, Systemd)	`tar` + `rsync`	Weekly	3 Months	Offsite S3/B2

8.2.2 Backup Script Implementation (`backup_db.sh`)

```
Bash

#!/bin/bash
# /opt/monitoring-app/scripts/backup_db.sh
set -euo pipefail

BACKUP_DIR="/var/backups/postgres"
DATE=$(date +%Y-%m-%d_%H%M%S)
```

```
FILENAME="gpu_monitor_${DATE}.dump"

# 1. Dump database (includes TimescaleDB hypertables and
continuous aggregates)
pg_dump -U postgres -Fc -f "${BACKUP_DIR}/${FILENAME}"
gpu_monitor_db

# 2. Compress
gzip "${BACKUP_DIR}/${FILENAME}"

# 3. Sync to offsite storage (e.g., Backblaze B2)
rclone copy "${BACKUP_DIR}/${FILENAME}.gz" b2:gpu-monitor-
backups/db/

# 4. Local cleanup (keep last 7 days locally to save VPS disk
space)
find "${BACKUP_DIR}" -type f -name "*.dump.gz" -mtime +7 -
delete
```

8.2.3 Automated Restore Testing (KB Sec 11 Requirement)

Backups are useless if they cannot be restored. A monthly automated test ensures validity.

1. **Trigger:** A cron job on the 1st of the month spins up a temporary, cheap VPS (e.g., via DigitalOcean/Hetzner API).
2. **Action:** The script installs PostgreSQL, pulls the latest backup from S3/B2, and runs `pg_restore`.
3. **Validation:** It runs `python manage.py check --deploy` and executes a basic `SELECT count(*) FROM rigs_rig;` to verify data integrity.
4. **Teardown:** The temporary VPS is destroyed.
5. **Alerting:** If the validation fails, an alert is sent to the system administrator via email/webhook.

8.3 Monitoring the Monitor (Meta-Monitoring)

The platform must not be a black box. We implement a lightweight, external probing strategy to avoid the overhead of running a full Prometheus/Grafana stack on the same VPS.

8.3.1 Internal Health Endpoint (`/api/v1/health/`)

As defined in Sec 4.8.3, this unauthenticated endpoint returns a JSON payload detailing the internal state of the application.

```
Json

{
  "status": "healthy",
  "version": "1.2.0",
```

```
  "uptime_seconds": 864000,
  "db_connection": "ok",
  "db_migration_status": "up_to_date",
  "timescale_version": "2.14.1",
  "active_rigs": 842,
  "ingest_qps_1m": 14.2
}
```

Implementation Detail: The view caches the response for 10 seconds to prevent external probes from generating unnecessary database load.

8.3.2 External Probing Strategy

Probe Target	Tool	Frequency	Alert Threshold
HTTPS & TLS Cert	UptimeRobot / HetrixTools	60s	HTTP != 200, Cert expires < 14 days
Health Endpoint	UptimeRobot (JSON parser)	60s	`status != "healthy"` or `db_connection != "ok"`
Ingest Endpoint	Custom Script (Cron)	5m	Fails to submit a dummy payload (tests DB write path)
Host Resources	Netdata (Cloud) or HetrixTools Agent	60s	CPU > 90% for 5m, Disk > 85%, RAM > 90%

8.4 Logging Strategy & Observability

Logs are structured for machine parsing and forwarded to a central location if needed, but primarily retained locally with strict rotation.

8.4.1 Django Logging Configuration (`settings.py`)

Uses `python-json-logger` to output structured JSON, making it easy to grep or ship to tools like Datadog/Loki later.

```
Python

LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
```

```
    'formatters': {
        'json': {
            '()': 'pythonjsonlogger.jsonlogger.JsonFormatter',
            'format': '%(asctime)s %(levelname)s %(name)s
%(message)s %(correlation_id)s'
        },
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'json',
        },
    },
    'root': {
        'handlers': ['console'],
        'level': 'INFO',
    },
    'loggers': {
        'django': {'level': 'INFO'},
        'apps.metrics': {'level': 'DEBUG'}, # Detailed
ingestion logging
    },
}
```

8.4.2 Correlation IDs

A custom middleware (`CorrelationIdMiddleware`) generates a UUID for every incoming request (or reads it from the `X-Correlation-ID` header if the agent provides one). This ID is injected into the thread-local context and appended to every log line, allowing operators to trace a specific payload from Nginx access logs → Django ingestion view → PostgreSQL write.

8.4.3 Log Rotation

Managed by `logrotate` to prevent disk exhaustion.

```
Text

# /etc/logrotate.d/monitoring-app
/var/log/monitoring/*.log {
    daily
    missingok
```

```
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 monitoring adm
    sharedscripts
    postrotate
        systemctl reload gunicorn > /dev/null 2>/dev/null ||
true
    endscript
}
```

8.5 Upgrade & Migration Procedures

Upgrades must be executed without dropping agent payloads or interrupting dashboard users.

8.5.1 Server Deployment Pipeline (Zero-Downtime)

Executed via a simple bash script or CI/CD runner (e.g., GitHub Actions via SSH).

```
Bash

#!/bin/bash
# deploy.sh
set -e

cd /opt/monitoring-app
```

```
git pull origin main

# 1. Install dependencies
./venv/bin/pip install -r requirements.txt

# 2. Apply database migrations (TimescaleDB & Django)
# Note: Heavy TimescaleDB operations should be done
concurrently if possible,
# but for <1000 rigs, standard migrate is fast enough.
./venv/bin/python manage.py migrate --noinput

# 3. Collect static files for Nginx
./venv/bin/python manage.py collectstatic --noinput

# 4. Graceful reload of Gunicorn (Zero dropped requests)
systemctl reload gunicorn

# 5. Restart background scheduler if logic changed
systemctl restart monitoring-cron
```

8.5.2 Schema Migration Safety Rules

1. **No Destructive Changes:** Never drop a column or rename a field in a single deploy.
   - *Step 1:* Add new column, deploy.
   - *Step 2:* Update code to write to both, deploy.
   - *Step 3:* Backfill data, deploy.
   - *Step 4:* Update code to read from new, deploy.
   - *Step 5:* Drop old column, deploy.
2. **TimescaleDB Hypertables:** Adding a column to a hypertable is fast and non-blocking. Changing a column type requires creating a new column and migrating data.

8.5.3 Agent Rollout Strategy

The agent is decoupled from the server.

1. **Backward Compatibility:** The server *must* support the current agent version (`schema_version: "1.0"`) before the new agent (`schema_version: "1.1"`) is distributed.
2. **Distribution:** Agents are updated via standard OS package managers (`apt upgrade monitoring-agent`) or a centralized configuration management tool (Ansible).
3. **Tracking:** The dashboard highlights rigs running outdated agent versions (using the `last_agent_version` field in the `rigs_rig` table), allowing operators to target stragglers.

9. PERFORMANCE & SCALING ANALYSIS

This section provides the mathematical and architectural proof that the single Ubuntu VPS design can reliably handle the target ceiling of **~1,000 rigs** reporting at 1-minute intervals. It defines payload budgets, database throughput limits, query performance targets, and exact VPS hardware sizing, while also outlining the escape hatch for horizontal scaling if the fleet grows beyond the initial scope.

9.1 Payload Size & Network Bandwidth Estimates

To prevent network saturation and minimize agent CPU overhead, payload sizes are strictly budgeted. The agent (Sec 3) collects extensive telemetry, but the JSON structure is optimized for density.

9.1.1 Payload Composition Budget

Payload Section	Uncompressed Size (Est.)	Compressed Size (gzip, Est.)	Notes
Headers & Envelope	~200 Bytes	~150 Bytes	UUID, timestamps, schema version
Inventory (Static)	~2.5 KB	~600 Bytes	CPU, Mobo, GPU specs (highly compressible text)
Metrics (Dynamic)	~1.5 KB	~400 Bytes	Utilizations, temps, arrays of GPU stats
Software & Docker	~1.5 KB	~450 Bytes	OS info, container states
Errors (Latest)	~1.0 KB	~300 Bytes	Deduplicated journalctl/systemd logs
Total per Payload	**~6.7 KB**	**~1.9 KB**	**Target: < 2.0 KB compressed**

9.1.2 Network Ingress Calculations

- **Average Load:** 1,000 rigs / 60 seconds = **16.67 requests/sec (RPS)**.
- **Burst Load:** Cron jobs naturally cluster. Assuming a 3x burst factor + jitter = **~50 RPS peak**.
- **Peak Bandwidth:** 50 RPS × 1.9 KB = **95 KB/s** (approx. **0.76 Mbps**).
- **Conclusion:** Network ingress is negligible. A standard 1 Gbps VPS uplink will utilize less than 0.1% of its capacity, leaving ample room for dashboard egress.

9.2 Write Throughput & Database IOPS

The ingestion pipeline (Sec 4.3) performs multiple database operations per payload. We must prove that PostgreSQL and TimescaleDB can sustain this write load on a single node.

9.2.1 Write Operations per Ingest Request

Target Table	Operation	Purpose
`rigs_rig`	`UPDATE`	Bump `last_seen` timestamp, update `status`.
`metrics_latest_snapshot`	`UPSERT`	Overwrite current state for instant dashboard loading.

Target Table	Operation	Purpose
`metrics_timeseries`	`INSERT`	Append to TimescaleDB hypertable for historical charts.
`metrics_latest_errors`	`UPSERT`	Update error counts or insert new deduplicated errors.
Total	**~4 to 8 writes**	Depending on error payload size.

9.2.2 Peak Write Throughput

- **Peak RPS:** 50 requests/sec.
- **Peak DB Writes:** 50 RPS × 8 writes = **400 writes/sec**.
- **PostgreSQL Capacity:** A modern NVMe-backed PostgreSQL instance can comfortably sustain **2,000 to 5,000+ writes/sec** (depending on row size and index count).
- **Conclusion:** The write load is well within the safety margin (utilizing ~10-20% of peak DB capacity). TimescaleDB's native batch-insert optimizations further reduce the I/O overhead for the `metrics_timeseries` hypertable.

9.3 Query Performance Budgets

The dashboard (Sec 5) relies on HTMX polling every 30 seconds. Queries must execute rapidly to ensure the VPS CPU is not exhausted by concurrent dashboard users.

9.3.1 Fleet Overview (Rig List)

- **Challenge:** Rendering 1,000 rows every 30 seconds via HTMX generates a massive HTML payload (~200 KB+) and heavy DOM diffing on the client.
- **Optimization: Server-Side Pagination**. The fleet list is restricted to **50 rigs per page**.
- **Query:** `SELECT ... FROM rigs_rig JOIN metrics_latest_snapshot ... LIMIT 50 OFFSET X`
- **Execution Time:** `< 5ms` (using B-Tree indexes on `owner_id` and `status`).
- **HTMX Payload Size:** ~10 KB per page swap.

9.3.2 Rig Detail (Historical Charts)

- **Challenge:** Fetching 24 hours of minute-level data (1,440 points) for 8 different charts.
- **Optimization:** Querying TimescaleDB Continuous Aggregates (Sec 6.3.1) for ranges > 24H, and using native hypertable chunk exclusion for recent data.
- **Query Execution:** `< 20ms` per chart.
- **JSON Payload:** 1,440 points × 8 bytes = ~11 KB per chart. Total chart payload `< 100 KB`.
- **Browser Rendering:** Chart.js / uPlot renders 1,440 points in `< 50ms` on modern hardware.

9.3.3 Concurrency Budget

- Assuming 10 concurrent dashboard users polling 3 pages each per minute.
- Total dashboard queries: ~30 queries/min (0.5 QPS).
- **Conclusion:** Dashboard read load is statistically insignificant compared to the agent write load.

9.4 Resource Sizing (VPS Tiers)

Based on the throughput calculations, the following hardware specifications are required to maintain a stable, responsive environment for 1,000 rigs.

9.4.1 Recommended VPS Specification

Resource	Specification	Justification
CPU	**4 vCPUs**	Gunicorn formula: `(2 * Cores) + 1` = 9 workers. 4 vCPUs easily handle 50 RPS ingestion + background cron tasks + OS overhead.
RAM	**16 GB**	**PostgreSQL/TimescaleDB:** Requires ~4-6 GB for `shared_buffers` and caching recent hypertable chunks. **Gunicorn:** 9 workers × 150 MB = ~1.3 GB. **OS/Nginx:** ~1.5 GB. **Headroom:** Leaves ~7 GB for OS page cache and spike absorption.
Storage	**250 GB NVMe SSD**	**Data Growth:** ~5 GB/day (raw + indexes). **14-day Retention:** ~70 GB. **Backups/Logs:** ~50 GB. **OS/App:** ~20 GB. **Total:** ~140 GB used. 250 GB provides safe overhead. NVMe is **mandatory** for TimescaleDB write IOPS.
Network	**1 Gbps**	Peak ingress is < 1 Mbps. Egress for 10 dashboard users is < 5 Mbps.

9.4.2 Storage IOPS Requirements

- **Write IOPS:** 400 writes/sec (burst).
- **Read IOPS:** Minimal, mostly served from RAM (Postgres `shared_buffers`).
- **Requirement:** Standard SATA SSDs will bottleneck during chunk compression or vacuuming. **NVMe storage is a strict requirement** for the single-VPS architecture.

9.5 Horizontal Scaling Path (Beyond 1,000 Rigs)

While the requirements explicitly state a ceiling of ~1,000 rigs to maintain single-VPS simplicity, a robust architecture must document the escape hatch if the fleet grows to 5,000 or 10,000 rigs.

9.5.1 The Bottleneck Shift

At 10,000 rigs (500 RPS peak, 4,000 writes/sec), the single PostgreSQL instance will hit CPU and I/O limits during TimescaleDB chunk compression and continuous aggregate refreshes.

9.5.2 Evolution Steps

Phase	Fleet Size	Architectural Change	Impact
Phase 1	1,000 - 2,500	**Read Replicas**	Add a managed Postgres Read Replica. Route all Dashboard (HTMX) queries to the replica. Keep Ingestion on the primary.
Phase 2	2,500 - 5,000	**Ingestion Queue**	Decouple HTTP from DB. Nginx pushes payloads to Redis Streams or RabbitMQ. Python/Celery workers consume the queue and batch-insert into TimescaleDB.
Phase 3	5,000+	**Edge Proxies & Sharding**	Deploy lightweight Go/Rust ingest proxies at the edge to validate API keys and pre-parse JSON. Shard TimescaleDB by `rig_uuid` or geographic region.

Implementation Note for v1: The Django app is structured (Sec 4.1) so that the `metrics` app can be extracted into a microservice, and the `IngestView` can be swapped from a direct DB write to a queue publisher without altering the agent contract.

10. TESTING STRATEGY

This section defines the quality assurance blueprint for the platform. Given the system's reliance on external hardware states (which are inherently flaky) and strict performance

budgets (Sec 9), the testing strategy prioritizes **fault-injection at the unit level**, **database-level idempotency proofs**, and **realistic load simulation**.

We adopt a pragmatic test pyramid, avoiding brittle UI tests in favor of robust API and data-layer validation.

10.1 Unit Testing (Component Isolation)

Unit tests focus on the internal logic of the Agent and Server, heavily utilizing mocking to simulate hardware states and network conditions.

10.1.1 Agent Unit Tests (`pytest` + `unittest.mock`)

The agent must never crash due to missing hardware or hanging subprocesses. Tests enforce the "partial failure tolerance" requirement (KB Sec 2.5).

Test Scenario	Mocking Strategy	Expected Assertion
Happy Path	Mock `psutil`, `pynvml`, `subprocess` to return valid data.	Payload matches JSON Schema v1.0. All fields populated.
GPU Driver Missing	`pynvml.nvmlInit()` raises `NVMLError_DriverNotLoaded`.	Agent logs warning. `metrics.gpus` is an empty array `[]`. Payload is still sent.
SMART/Storage Fallback	`subprocess.run("smartctl")` raises `CalledProcessError`.	`storage[*].smart_health` is `null`. Agent does not abort.
Network Timeout	`requests.post()` raises `ConnectionError` or `Timeout`.	Retry logic triggers. Exponential backoff (1s, 2s, 4s) is verified via mocked `time.sleep`.
Hard Timeout (Cron Guard)	Mock a collector to `time.sleep(60)`.	`signal.alarm(45)` triggers `TimeoutError`. Agent catches it, logs, and exits cleanly before the next cron minute.

10.1.2 Server Unit Tests (`pytest-django`)

Focuses on DRF serializers, RBAC, and background logic.

Test Scenario	Target Component	Expected Assertion

Test Scenario	Target Component	Expected Assertion
Serializer Validation	`IngestSerializer`	Rejects payloads with missing `rig_uuid` or invalid `schema_version`. Accepts payloads with unknown extra fields (forward compatibility).
Ownership Enforcement	`RigViewSet`	User A requesting User B's `rig_uuid` receives `404 Not Found` (prevents enumeration) or `403 Forbidden`.
Offline Detection Logic	`update_rig_status` cmd	Mock `timezone.now()`. Rigs with `last_seen` > 10 mins ago are strictly marked `offline`.
API Key Hashing	`ApiKey` model	Saving a new key stores an `argon2id` hash. The plaintext is never written to the DB.

10.2 Integration Testing (Pipeline & Database)

Integration tests verify that the Django ORM, DRF views, and TimescaleDB extensions work together correctly. These tests require a real PostgreSQL database with the TimescaleDB extension enabled in the CI environment.

10.2.1 Idempotency & Conflict Resolution

The most critical integration test proves that network retries do not corrupt time-series data.

```
Python

# tests/integration/test_ingest_idempotency.py
def test_duplicate_payload_is_ignored(api_client,
valid_payload, db):
    # 1. First submission
    response1 = api_client.post('/api/v1/ingest/',
valid_payload, format='json')
    assert response1.status_code == 200
    assert MetricSnapshot.objects.count() == 1

    # 2. Exact same payload (simulating agent retry)
    response2 = api_client.post('/api/v1/ingest/',
valid_payload, format='json')
    assert response2.status_code == 202  # Accepted
(Duplicate)
    assert MetricSnapshot.objects.count() == 1  # DB state
unchanged
```

10.2.2 TimescaleDB Hypertable & Aggregation Verification

- **Migration Tests:** Assert that `python manage.py migrate` successfully executes `create_hypertable()` and creates the continuous aggregate materialized views.
- **Chunk Exclusion:** Verify that querying a specific time range only scans the relevant TimescaleDB chunks (verified via `EXPLAIN ANALYZE` in test assertions).
- **Error Deduplication:** Send the same kernel error message 5 times. Assert `metrics_latest_errors` only contains 1 row, but `occurrence_count` equals 5.

10.3 End-to-End (E2E) Testing

E2E tests simulate the complete user journey, from API key generation to dashboard visualization. We use **Playwright** (Python bindings) because it handles HTMX DOM swaps and asynchronous network requests better than legacy tools like Selenium.

10.3.1 The "Golden Path" E2E Scenario

1. **Enrollment:** Playwright logs in as a test user, navigates to API Keys, and clicks "Generate". Asserts the plaintext key is visible in the DOM exactly once.
2. **Agent Simulation:** A background Python thread uses the generated key to send 3 valid payloads over 3 minutes.
3. **Fleet Overview:** Playwright navigates to `/dashboard/`. Asserts the new rig appears in the table with a "Online" badge.
4. **Detail View & Charts:** Playwright clicks the rig, switches to the "Historical Charts" tab. Waits for the `hx-trigger="revealed"` event. Asserts that the Chart.js `<canvas>` elements are rendered and contain data points.
5. **Stale Detection:** The agent thread is killed. Playwright waits for the server's stale threshold (e.g., 3 minutes in test config). Asserts the UI badge updates to "Stale" via HTMX polling *without* a full page reload.

10.4 Load Testing Approach

To validate the mathematical proofs in Section 9, we use **Locust** (Python) to simulate the target ceiling of 1,000 rigs.

10.4.1 Locust Script Design

```
Python

# locustfile.py
from locust import HttpUser, task, between
import json, uuid, random

class RigAgent(HttpUser):
    wait_time = between(55, 65) # Simulate cron jitter around
60s

    def on_start(self):
        self.rig_uuid = str(uuid.uuid4())
        self.api_key = "test-load-key" # Pre-provisioned in DB

    @task
    def send_telemetry(self):
        payload =
generate_randomized_v1_payload(self.rig_uuid)
        headers = {"X-API-Key": self.api_key, "Content-
Encoding": "gzip"}
        self.client.post("/api/v1/ingest/",
data=json.dumps(payload), headers=headers)
```

10.4.2 Execution & Success Criteria

- **Environment:** Run Locust on a separate machine to avoid starving the VPS of CPU.
- **Ramp-up:** Add 50 users per second until reaching 1,000 concurrent agents.
- **Pass/Fail Metrics:**
  - **Throughput:** Sustains ~16.67 RPS average, handling bursts up to 50 RPS.
  - **Latency:** 95th percentile response time for `/api/v1/ingest/` remains `< 200ms`.
  - **Error Rate:** `< 0.1%` (excluding intentional 429 Rate Limit triggers).
  - **Resource Saturation:** VPS CPU does not exceed 85%; PostgreSQL active connections remain stable (validating connection pooling).

10.5 Contract Testing for Schema Evolution

To guarantee the "backward compatibility" requirement (KB Sec 7), we implement strict contract testing using a **Golden Payload Repository**.

10.5.1 The Golden Payload Repository

A directory in the codebase (`tests/contracts/payloads/`) contains static JSON files representing every supported version of the agent payload:

- `v1.0_minimal.json` (Only required fields)
- `v1.0_full.json` (All optional fields populated)
- `v1.1_with_ai_metrics.json` (Future version)

10.5.2 CI/CD Contract Enforcement

A dedicated pytest suite iterates through this repository on every PR:

```
Python

# tests/contracts/test_schema_evolution.py
import pytest, json, glob

@pytest.mark.parametrize("payload_file",
glob.glob("tests/contracts/payloads/*.json"))
def test_server_accepts_all_supported_versions(api_client,
payload_file, db):
    with open(payload_file) as f:
        payload = json.load(f)

    response = api_client.post('/api/v1/ingest/', payload,
format='json')

    # Server must NEVER return 400 Bad Request for a known
historical schema
    assert response.status_code in [200, 202], f"Server broke
backward compatibility for {payload_file}"
```

Rule: If a developer changes the `IngestSerializer` in a way that breaks `v1.0_minimal.json`, the CI pipeline fails, blocking the deployment.

11. APPENDICES & FORMAL AUDIT

This final section consolidates reference specifications, operational templates, and a formal requirements-to-architecture audit. It serves as the implementation-ready annex and the approval gate for the blueprint.

□ Appendix A: Full JSON Schema Definition (Agent Payload v1.0)

Validates against `draft/2020-12/schema`. Enforces strict typing, nullability rules, and forward-compatibility (`additionalProperties: true` at root).

```
Json

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "GPU Rig Monitor Ingest Payload v1.0",
  "type": "object",
  "required": ["rig_uuid", "schema_version", "agent_version",
"timestamp", "inventory", "metrics", "software"],
  "additionalProperties": true,
  "properties": {
    "rig_uuid": { "type": "string", "format": "uuid" },
    "schema_version": { "type": "string", "enum": ["1.0"] },
    "agent_version": { "type": "string", "pattern":
"^\\d+\\.\\d+\\.\\d+$" },
    "timestamp": { "type": "string", "format": "date-time" },

    "inventory": {
      "type": "object",
      "properties": {
        "cpu": {
          "type": "object",
          "properties": {
            "model": { "type": "string" }, "vendor": { "type":
"string" }, "arch": { "type": "string" },
            "sockets": { "type": "integer" },
"physical_cores": { "type": "integer" }, "logical_cores": {
"type": "integer" }
          }
        },
        "memory": {
          "type": "object",
          "properties": {
            "total_bytes": { "type": "integer" },
"swap_total_bytes": { "type": "integer" }
          }
        },
        "motherboard": {
          "type": "object",
          "properties": {
```

```
            "manufacturer": { "type": ["string", "null"] },
"model": { "type": ["string", "null"] }, "bios_version": {
"type": ["string", "null"] }
          }
        },
        "storage": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "device": { "type": "string" }, "model": {
"type": ["string", "null"] }, "serial": { "type": ["string",
"null"] },
              "capacity_bytes": { "type": "integer" },
"smart_health": { "type": ["string", "null"] },
"nvme_wear_level": { "type": ["integer", "null"] }
            }
          }
        },
        "network": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "interface": { "type": "string" }, "ipv4": {
"type": ["string", "null"] }, "link_speed_mbps": { "type":
["integer", "null"] }
            }
          }
        },
        "gpus": {
          "type": "array",
          "items": {
            "type": "object",
            "required": ["uuid", "model", "mem_total_mb"],
            "properties": {
              "uuid": { "type": "string" }, "pci_bus_id": {
"type": "string" }, "model": { "type": "string" },
              "mem_total_mb": { "type": "integer" },
"pcie_gen": { "type": ["integer", "null"] }, "pcie_width": {
"type": ["integer", "null"] }
            }
          }
        }
      }
    },

    "metrics": {
      "type": "object",
      "properties": {
        "cpu": {
```

```
          "type": "object",
          "properties": {
            "load_avg": { "type": "number" },
"utilization_pct": { "type": "number", "minimum": 0,
"maximum": 100 },
            "temp_c": { "type": ["number", "null"] }
          }
        },
        "memory": {
          "type": "object",
          "properties": {
            "used_bytes": { "type": "integer" }, "free_bytes":
{ "type": "integer" }, "cached_bytes": { "type": "integer" },
            "swap_used_bytes": { "type": "integer" }
          }
        },
        "storage": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "device": { "type": "string" }, "usage_pct": {
"type": "number" }, "mount_state": { "type": "string" },
              "temp_c": { "type": ["number", "null"] },
"nvme_media_errors": { "type": ["integer", "null"] }
            }
          }
        },
        "network": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "interface": { "type": "string" }, "rx_bytes": {
"type": "integer" }, "tx_bytes": { "type": "integer" },
              "rx_errors": { "type": "integer" }, "tx_errors":
{ "type": "integer" }
            }
          }
        },
        "gpus": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "uuid": { "type": "string" }, "mem_used_mb": {
"type": "integer" }, "mem_free_mb": { "type": "integer" },
              "mem_util_pct": { "type": "number" },
"gpu_util_pct": { "type": "number" }, "temp_c": { "type":
["number", "null"] },
```

```
              "fan_speed_pct": { "type": ["number", "null"] },
"power_draw_w": { "type": ["number", "null"] },
              "power_limit_w": { "type": ["number", "null"] },
"pcie_throughput_gbps": { "type": ["number", "null"] },
              "processes": {
                "type": "array",
                "items": {
                  "type": "object",
                  "properties": { "pid": { "type": "integer"
}, "name": { "type": "string" }, "mem_mb": { "type": "integer"
} }
                }
              }
            }
          }
        },
        "ai_heuristics": {
          "type": "object",
          "properties": {
            "fragmentation_risk": { "type": "string", "enum":
["low", "medium", "high", "unknown"] },
            "expected_gpu_count": { "type": "integer" },
"actual_gpu_count": { "type": "integer" },
            "gpu_mismatch": { "type": "boolean" }
          }
        },
        "docker_containers": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "name": { "type": "string" }, "image": { "type":
"string" }, "status": { "type": "string" },
              "restart_count": { "type": "integer" },
"uptime_seconds": { "type": ["integer", "null"] }
            }
          }
        }
      }
    },

    "software": {
      "type": "object",
      "properties": {
        "hostname": { "type": "string" }, "os_distro": {
"type": "string" }, "kernel": { "type": "string" },
        "uptime_s": { "type": "integer" }, "ntp_sync": {
"type": "boolean" }, "nvidia_driver": { "type": ["string",
"null"] },
        "cuda_version": { "type": ["string", "null"] },
"docker_version": { "type": ["string", "null"] },
```

```
        "reboot_required": { "type": "boolean" }
      }
    },

    "errors": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["source", "message", "timestamp"],
        "properties": {
          "source": { "type": "string", "enum": ["kernel",
"nvidia", "docker", "systemd", "agent"] },
          "message": { "type": "string", "maxLength": 1024 },
          "timestamp": { "type": "string", "format": "date-
time" },
          "count": { "type": "integer", "minimum": 1 }
        }
      }
    }
  }
}
```

□ Appendix B: OpenAPI 3.0 Contract Summary

Implementation-ready endpoint catalog. Full Swagger/Redoc YAML can be auto-generated from DRF using `drf-spectacular`.

Method	Path	Auth	Purpose	Request Schema	Response Schema	Status Codes
POST	/api/v1/ingest/	X-API-Key header	Telemetry submission	Appendix A JSON	{status, message, next_expected}	200 (new), 202 (dup), 400, 401, 429, 5xx
GET	/api/v1/health/	None	Internal platform health	None	{status, version, uptime_s, db_connection, active_rigs, ingest_qps	200, 503

Method	Path	Auth	Purpose	Request Schema	Response Schema	Status Codes
					}	
GET	/dashboard/rigs/	Session Cookie	Fleet overview (paginated)	Query: ?status=&tag=&search=&page=	HTML (Django Template)	200, 403
GET	/dashboard/rigs/{uuid}/	Session Cookie	Rig detail page	Path: uuid	HTML (Tabbed layout)	200, 404
GET	/dashboard/rigs/{uuid}/htmx-metrics/	Session Cookie	30s live poll partial	Header: HX-Request: true	HTML fragment (<tr> or metric cards)	200, 404
GET	/dashboard/rigs/{uuid}/chart-data/	Session Cookie	Historical chart JSON	Query: ?metric=gpu_temp&range=24h	JSON {labels: [], datasets: []}	200, 400, 404
POST	/accounts/api-keys/	Session Cookie	Generate/Revoke API key	`{name, action: "create"	"revoke"}`	HTML partial (key reveal or row update)	

Global Headers:

- `Content-Type: application/json` (API)
- `X-Correlation-ID: <uuid>` (Injected by middleware, echoed in response)
- `Cache-Control: no-cache` (HTMX polling endpoints)

□ Appendix C: Production Systemd & Cron Templates

C.1 Gunicorn Application Service

```
Ini

# /etc/systemd/system/gunicorn.service
[Unit]
Description=GPU Monitor Django Application
After=network.target postgresql.service
Wants=postgresql.service

[Service]
User=monitoring
Group=monitoring
WorkingDirectory=/opt/monitoring-app
EnvironmentFile=/opt/monitoring-app/.env
ExecStart=/opt/monitoring-app/venv/bin/gunicorn \
    --workers 4 \
    --bind unix:/run/gunicorn/gunicorn.sock \
    --access-logfile - \
    --error-logfile /var/log/monitoring/gunicorn.error.log \
    config.wsgi:application
ExecReload=/bin/kill -s HUP $MAINPID
Restart=on-failure
RestartSec=5
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

C.2 Agent Cron Job (with overlap protection & resource guardrails)

```
Bash

# /etc/cron.d/monitoring-agent
* * * * * monitoring-agent flock -n /var/lock/monitoring-
agent.lock \
    nice -n 19 ionice -c 3 \
    /opt/monitoring-agent/venv/bin/python /opt/monitoring-
agent/run.py \
    >> /var/log/monitoring-agent/cron.log 2>&1
```

C.3 Backup Systemd Timer & Service

```ini
# /etc/systemd/system/monitoring-backup.service
[Unit]
Description=GPU Monitor Database Backup
After=postgresql.service

[Service]
Type=oneshot
User=monitoring
ExecStart=/opt/monitoring-app/scripts/backup_db.sh
StandardOutput=append:/var/log/monitoring/backup.log
StandardError=append:/var/log/monitoring/backup.log

# /etc/systemd/system/monitoring-backup.timer
[Unit]
Description=Daily DB Backup Timer

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

□ Appendix D: Glossary & Acronyms

Term	Definition
HDA	Hypermedia-Driven Application. UI pattern where server-rendered HTML fragments drive state changes via HTMX, avoiding SPA complexity.
Hypertable	TimescaleDB abstraction that automatically partitions time-series data into chunks based on time intervals.
Continuous Aggregate	TimescaleDB materialized view that incrementally computes aggregations (hourly/daily) as new data arrives.
Idempotency	Property where repeating the same request yields the same system state without side effects (enforced via `UNIQUE` constraints + `ON CONFLICT`).
RBAC	Role-Based Access Control. Restricts dashboard/data access to `Owner`, `Admin`, or `Viewer` roles.
STRIDE	Threat modeling framework: Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege.

Term	Definition
Schema Versioning	Protocol where `schema_version` in payloads allows the server to route validation logic, ensuring backward/forward compatibility during rollouts.
Morphdom	HTMX extension that performs DOM diffing during swaps, preserving scroll position and input focus during 30s polling.

Agent Repository (`gpu-monitor-agent`)

Directory Tree:

```
Text

gpu-monitor-agent/
├── .github/workflows/
├── src/
│   └── agent/
│       ├── __init__.py
│       ├── main.py             # Entry point & cron execution
logic
│       ├── config.py           # YAML parsing & UUID lifecycle
│       ├── collectors/         # Modular metric gatherers
│       │   ├── cpu.py
│       │   ├── gpu.py          # pynvml logic
│       │   ├── storage.py
│       │   ├── network.py
│       │   └── docker.py
│       ├── transport.py        # Requests, retry, gzip logic
│       └── schema.py           # Payload construction
├── tests/
│   ├── conftest.py             # Mocks for psutil/pynvml
│   └── test_collectors.py
├── deploy/
│   ├── monitoring-agent.service  # Systemd unit
│   └── cron.d/monitoring-agent   # Cron template
├── pyproject.toml              # Modern Python packaging
└── README.md
```

Server Repository (`gpu-monitor-server`)

Directory Tree:

```
Text

gpu-monitor-server/
├── .github/workflows/        # CI/CD pipelines
├── config/                   # Django project settings
│   ├── settings/
│   │   ├── base.py
│   │   ├── dev.py
│   │   ├── prod.py
│   │   └── test.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── apps/                     # Domain-driven Django apps
│   ├── accounts/             # Users, API Keys, RBAC
│   ├── rigs/                 # Inventory, Tags, Ownership
│   ├── metrics/              # Ingestion, TimescaleDB models
│   ├── dashboard/            # HTMX views, templates
│   └── audit/                # Immutable event logging
├── templates/                # Global Django templates
├── static/                   # CSS, JS, HTMX, Chart.js
├── tests/                    # Pytest suites
│   ├── contracts/            # Golden Payload JSONs (Sec
10.5)
│   ├── integration/
│   └── unit/
├── scripts/                  # Backup & maintenance bash
scripts
├── docker-compose.yml        # Local dev environment
├── Makefile                  # Developer shortcuts
├── manage.py
└── requirements/
    ├── base.txt
    ├── dev.txt
    └── prod.txt
```